

0. Setup: Gathering Our Tools

```
!pip install "qc-grader[qiskit,jupyter] @ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git"
```

```

Collecting qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git (from qc-grader[jupyter])
  Cloning https://github.com/qiskit-community/Quantum-Challenge-Grader.git to /tmp/pip-install-uz_s5slm/qc-grader
  Running command git clone --filter=blob:none --quiet https://github.com/qiskit-community/Quantum-Challenge-Grader.git
  Resolved https://github.com/qiskit-community/Quantum-Challenge-Grader.git to commit 1d7a6915623b0cfeac4c114391
  Preparing metadata (setup.py) ... done
Requirement already satisfied: typeguard in /usr/local/lib/python3.11/dist-packages (from qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Collecting jsonpickle==3.0.3 (from qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
  Downloading jsonpickle-3.0.3-py3-none-any.whl.metadata (7.3 kB)
Requirement already satisfied: requests==2.32.3 in /usr/local/lib/python3.11/dist-packages (from qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Collecting ipycytoscape (from qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
  Downloading ipycytoscape-1.3.3-py2.py3-none-any.whl.metadata (7.5 kB)
Requirement already satisfied: plotly in /usr/local/lib/python3.11/dist-packages (from qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Collecting networkx==3.2.1 (from qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
  Downloading networkx-3.2.1-py3-none-any.whl.metadata (5.2 kB)
Requirement already satisfied: graphviz in /usr/local/lib/python3.11/dist-packages (from qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Collecting ibm-platform-services==0.66.1 (from qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
  Downloading ibm_platform_services-0.66.1-py3-none-any.whl.metadata (9.0 kB)
Collecting ibm_cloud_sdk_core<4.0.0,>=3.24.1 (from ibm-platform-services==0.66.1->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
  Downloading ibm_cloud_sdk_core-3.24.2-py3-none-any.whl.metadata (8.7 kB)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests==2.32.3->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests==2.32.3->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests==2.32.3->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests==2.32.3->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Collecting qiskit~=2.1.0 (from qiskit[visualization]~=2.1.0; extra == "qiskit"->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
  Downloading qiskit-2.1.1-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (12 kB)
Collecting qiskit-ibm-runtime (from qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
  Downloading qiskit_ibm_runtime-0.41.0-py3-none-any.whl.metadata (21 kB)
Collecting qiskit-aer (from qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
  Downloading qiskit_aer-0.17.1-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (8.3 kB)
Collecting qiskit_serverless (from qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
  Downloading qiskit_serverless-0.25.4-py3-none-any.whl.metadata (5.5 kB)
Collecting jupyterlab (from qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
  Downloading jupyterlab-4.4.5-py3-none-any.whl.metadata (16 kB)
Requirement already satisfied: ipykernel in /usr/local/lib/python3.11/dist-packages (from qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Collecting rustworkx>=0.15.0 (from qiskit~=2.1.0->qiskit[visualization]~=2.1.0; extra == "qiskit"->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
  Downloading rustworkx-0.17.1-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (10 kB)
Requirement already satisfied: numpy<3,>=1.17 in /usr/local/lib/python3.11/dist-packages (from qiskit~=2.1.0->qiskit[visualization]~=2.1.0; extra == "qiskit"->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Requirement already satisfied: scipy>=1.5 in /usr/local/lib/python3.11/dist-packages (from qiskit~=2.1.0->qiskit[visualization]~=2.1.0; extra == "qiskit"->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Requirement already satisfied: dill>=0.3 in /usr/local/lib/python3.11/dist-packages (from qiskit~=2.1.0->qiskit[visualization]~=2.1.0; extra == "qiskit"->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Collecting stevedore>=3.0.0 (from qiskit~=2.1.0->qiskit[visualization]~=2.1.0; extra == "qiskit"->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
  Downloading stevedore-5.4.1-py3-none-any.whl.metadata (2.3 kB)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.11/dist-packages (from qiskit~=2.1.0->qiskit[visualization]~=2.1.0; extra == "qiskit"->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Requirement already satisfied: matplotlib>=3.3 in /usr/local/lib/python3.11/dist-packages (from qiskit[visualization]~=2.1.0; extra == "qiskit"->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Requirement already satisfied: pydot in /usr/local/lib/python3.11/dist-packages (from qiskit[visualization]~=2.1.0; extra == "qiskit"->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Requirement already satisfied: Pillow>=4.2.1 in /usr/local/lib/python3.11/dist-packages (from qiskit[visualization]~=2.1.0; extra == "qiskit"->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Collecting pylatexenc>=1.4 (from qiskit[visualization]~=2.1.0; extra == "qiskit"->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
  Downloading pylatexenc-2.10.0.tar.gz (162 kB)
  162.6/162.6 kB 4.0 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: seaborn>=0.9.0 in /usr/local/lib/python3.11/dist-packages (from qiskit[visualization]~=2.1.0; extra == "qiskit"->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Requirement already satisfied: sympy>=1.3 in /usr/local/lib/python3.11/dist-packages (from qiskit[visualization]~=2.1.0; extra == "qiskit"->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)
Requirement already satisfied: ipywidgets>=7.6.0 in /usr/local/lib/python3.11/dist-packages (from ipycytoscape->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader.git)

```

```
Collecting spectate>=1.0.0 (from ipycytoscape->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading spectate-1.0.1-py2.py3-none-any.whl.metadata (2.2 kB)
Requirement already satisfied: debugpy>=1.0 in /usr/local/lib/python3.11/dist-packages (from ipycytoscape->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: ipython>=7.23.1 in /usr/local/lib/python3.11/dist-packages (from ipycytoscape->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: jupyter-client>=6.1.12 in /usr/local/lib/python3.11/dist-packages (from ipycytoscape->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: matplotlib-inline>=0.1 in /usr/local/lib/python3.11/dist-packages (from ipycytoscape->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: nest-asyncio in /usr/local/lib/python3.11/dist-packages (from ipycytoscape->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from ipycytoscape->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: psutil in /usr/local/lib/python3.11/dist-packages (from ipycytoscape->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: pyzmq>=17 in /usr/local/lib/python3.11/dist-packages (from ipycytoscape->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: tornado>=6.1 in /usr/local/lib/python3.11/dist-packages (from ipycytoscape->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: traitlets>=5.1.0 in /usr/local/lib/python3.11/dist-packages (from ipycytoscape->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Collecting async-lru>=1.0.0 (from jupyterlab->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading async_lru-2.0.5-py3-none-any.whl.metadata (4.5 kB)
Requirement already satisfied: httpx>=0.25.0 in /usr/local/lib/python3.11/dist-packages (from jupyterlab->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: Jinja2>=3.0.3 in /usr/local/lib/python3.11/dist-packages (from jupyterlab->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: jupyter-core in /usr/local/lib/python3.11/dist-packages (from jupyterlab->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Collecting jupyter-lsp>=2.0.0 (from jupyterlab->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading jupyter_lsp-2.2.6-py3-none-any.whl.metadata (1.8 kB)
Collecting jupyter-server<3,>=2.4.0 (from jupyterlab->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading jupyter_server-2.16.0-py3-none-any.whl.metadata (8.5 kB)
Collecting jupyterlab-server<3,>=2.27.1 (from jupyterlab->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading jupyterlab_server-2.27.3-py3-none-any.whl.metadata (5.9 kB)
Requirement already satisfied: notebook-shim>=0.2 in /usr/local/lib/python3.11/dist-packages (from jupyterlab->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: setuptools>=41.1.0 in /usr/local/lib/python3.11/dist-packages (from jupyterlab->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.11/dist-packages (from jupyterlab->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: python-dateutil>=2.8.0 in /usr/local/lib/python3.11/dist-packages (from jupyterlab->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Collecting requests-ntlm>=1.1.0 (from qiskit-ibm-runtime->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading requests_ntlm-1.3.0-py3-none-any.whl.metadata (2.4 kB)
Requirement already satisfied: pydantic>=2.5.0 in /usr/local/lib/python3.11/dist-packages (from qiskit-ibm-runtime->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Collecting ray<3,>=2.30 (from ray[default]<3,>=2.30->qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading ray-2.48.0-cp311-cp311-manylinux2014_x86_64.whl.metadata (19 kB)
Requirement already satisfied: importlib-metadata<9,>=5.2.0 in /usr/local/lib/python3.11/dist-packages (from ray[default]<3,>=2.30->qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Collecting qiskit-ibm-runtime (from qiskit-ibm-runtime->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading qiskit_ibm_runtime-0.40.1-py3-none-any.whl.metadata (21 kB)
Collecting cloudpickle==2.2.1 (from qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading cloudpickle-2.2.1-py3-none-any.whl.metadata (6.9 kB)
Requirement already satisfied: tqdm<5,>=4.66.3 in /usr/local/lib/python3.11/dist-packages (from qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Collecting opentelemetry-api<1.33.1,>=1.18.0 (from qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading opentelemetry_api-1.33.0-py3-none-any.whl.metadata (1.6 kB)
Collecting opentelemetry-sdk<1.33.1,>=1.18.0 (from qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading opentelemetry_sdk-1.33.0-py3-none-any.whl.metadata (1.6 kB)
Collecting opentelemetry-exporter-otlp-proto-grpc<1.33.1,>=1.18.0 (from qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading opentelemetry_exporter_otlp_proto_grpc-1.33.0-py3-none-any.whl.metadata (2.5 kB)
Collecting opentelemetry-instrumentation-requests>=0.40b0 (from qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading opentelemetry_instrumentation_requests-0.57b0-py3-none-any.whl.metadata (2.6 kB)
Collecting ipywidgets>=7.6.0 (from ipycytoscape->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading ipywidgets-8.1.7-py3-none-any.whl.metadata (2.4 kB)
Collecting ipython>=7.23.1 (from ipycytoscape->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading ipython-8.37.0-py3-none-any.whl.metadata (5.1 kB)
Requirement already satisfied: pyarrow<19,>=16.0.0 in /usr/local/lib/python3.11/dist-packages (from ipycytoscape->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: aiohttp<4,>=3.10.0 in /usr/local/lib/python3.11/dist-packages (from ipycytoscape->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Collecting zipp==3.19.1 (from qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading zipp-3.19.1-py3-none-any.whl.metadata (3.5 kB)
Collecting certifi>=2017.4.17 (from requests==2.32.3->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading certifi-2024.7.4-py3-none-any.whl.metadata (2.2 kB)
Collecting s3fs>=2023.6.0 (from qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading s3fs-2025.7.0-py3-none-any.whl.metadata (1.4 kB)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp<4,>=3.10.0->qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp<4,>=3.10.0->qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp<4,>=3.10.0->qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.11/dist-packages (from aiohttp<4,>=3.10.0->qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.11/dist-packages (from aiohttp<4,>=3.10.0->qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp<4,>=3.10.0->qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp<4,>=3.10.0->qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: anyio in /usr/local/lib/python3.11/dist-packages (from httpx>=0.25.0->jupyterlab->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.11/dist-packages (from httpx>=0.25.0->jupyterlab->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.11/dist-packages (from httpcore==1.*->httpx>=0.25.0->jupyterlab->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
INFO: pip is looking at multiple versions of ibm-cloud-sdk-core to determine which version is compatible with ot
Collecting ibm_cloud_sdk_core<4.0.0,>=3.24.1 (from ibm-platform-services==0.66.1->qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading ibm_cloud_sdk_core-3.24.1-py3-none-any.whl.metadata (8.7 kB)
Requirement already satisfied: PyJWT<3.0.0,>=2.8.0 in /usr/local/lib/python3.11/dist-packages (from ibm-cloud-sdk-core==3.24.1->ibm-platform-services==0.66.1->qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
INFO: pip is looking at multiple versions of importlib-metadata to determine which version is compatible with ot
Collecting importlib-metadata<9,>=5.2.0 (from qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
  Downloading importlib_metadata-8.6.1-py3-none-any.whl.metadata (4.7 kB)
  Downloading importlib_metadata-8.6.0-py3-none-any.whl.metadata (4.7 kB)
  Downloading importlib_metadata-8.5.0-py3-none-any.whl.metadata (4.8 kB)
  Downloading importlib_metadata-8.4.0-py3-none-any.whl.metadata (4.7 kB)
Requirement already satisfied: decorator in /usr/local/lib/python3.11/dist-packages (from ipython>=7.23.1->ipykernel->qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
Collecting jedi>=0.16 (from ipython>=7.23.1->ipykernel->qiskit_serverless->qc-grader@ git+https://github.com/qiskit-community/Quantum-Challenge-Grader)
```

Downloading jedi-0.19.2-py2.py3-none-any.whl.metadata (22 kB)
Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.11/dist-packages (from ipython>=7.23.1->ipy
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in /usr/local/lib/python3.11/dist-packages (from ip
Requirement already satisfied: pygments>=2.4.0 in /usr/local/lib/python3.11/dist-packages (from ipython>=7.23.1-
Collecting stack_data (from ipython>=7.23.1->ipykernel->qc-grader@ git+<https://github.com/qiskit-community/Quantum-Challenge>)
Downloading stack_data-0.6.3-py3-none-any.whl.metadata (18 kB)
Collecting traitlets>=5.1.0 (from ipykernel->qc-grader@ git+<https://github.com/qiskit-community/Quantum-Challenge>)
Downloading traitlets-5.14.3-py3-none-any.whl.metadata (10 kB)
Collecting comm>=0.1.3 (from ipywidgets>=7.6.0->ipycytoscape->qc-grader@ git+<https://github.com/qiskit-community/Quantum-Challenge>)
Downloading comm-0.2.3-py3-none-any.whl.metadata (3.7 kB)
Collecting widgetsnbextension~4.0.14 (from ipywidgets>=7.6.0->ipycytoscape->qc-grader@ git+<https://github.com/qiskit-community/Quantum-Challenge>)
Downloading widgetsnbextension-4.0.14-py3-none-any.whl.metadata (1.6 kB)
Requirement already satisfied: jupyterlab_widgets~3.0.15 in /usr/local/lib/python3.11/dist-packages (from ipywi
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-packages (from jinja2>=3.0.3->j
Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python3.11/dist-packages (from jupyter-core->
Requirement already satisfied: argon2-cffi>=21.1 in /usr/local/lib/python3.11/dist-packages (from jupyter-server
Collecting jupyter-client>=6.1.12 (from ipykernel->qc-grader@ git+<https://github.com/qiskit-community/Quantum-Challenge>)
Downloading jupyter_client-8.6.3-py3-none-any.whl.metadata (8.3 kB)
Collecting jupyter-events>=0.11.0 (from jupyter-server<3,>=2.4.0->jupyterlab->qc-grader@ git+<https://github.com/qiskit-community/Quantum-Challenge>)
Downloading jupyter_events-0.12.0-py3-none-any.whl.metadata (5.8 kB)
Collecting jupyter-server-terminals>=0.4.4 (from jupyter-server<3,>=2.4.0->jupyterlab->qc-grader@ git+<https://github.com/qiskit-community/Quantum-Challenge>)
Downloading jupyter_server_terminals-0.5.3-py3-none-any.whl.metadata (5.6 kB)
Requirement already satisfied: nbconvert>=6.4.4 in /usr/local/lib/python3.11/dist-packages (from jupyter-server<
Requirement already satisfied: nbformat>=5.3.0 in /usr/local/lib/python3.11/dist-packages (from jupyter-server<3
Collecting overrides>=5.0 (from jupyter-server<3,>=2.4.0->jupyterlab->qc-grader@ git+<https://github.com/qiskit-community/Quantum-Challenge>)
Downloading overrides-7.7.0-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: prometheus-client>=0.9 in /usr/local/lib/python3.11/dist-packages (from jupyter-s
Requirement already satisfied: send2trash>=1.8.2 in /usr/local/lib/python3.11/dist-packages (from jupyter-server
Requirement already satisfied: terminado>=0.8.3 in /usr/local/lib/python3.11/dist-packages (from jupyter-server<
Requirement already satisfied: websocket-client>=1.7 in /usr/local/lib/python3.11/dist-packages (from jupyter-se
Requirement already satisfied: babel>=2.10 in /usr/local/lib/python3.11/dist-packages (from jupyterlab-server<3,
Collecting json5>=0.9.0 (from jupyterlab-server<3,>=2.27.1->jupyterlab->qc-grader@ git+<https://github.com/qiskit-community/Quantum-Challenge>)
Downloading json5-0.12.1-py3-none-any.whl.metadata (36 kB)
Requirement already satisfied: jsonschema>=4.18.0 in /usr/local/lib/python3.11/dist-packages (from jupyterlab-se
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3.3
Requirement already satisfied: cyclor>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3.3->q
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3.
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3.
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3.3
Collecting deprecated>=1.2.6 (from opentelemetry-api<1.33.1,>=1.18.0->qiskit_serverless->qc-grader@ git+<https://github.com/qiskit-community/Quantum-Challenge>)
Downloading Deprecated-1.2.18-py2.py3-none-any.whl.metadata (5.7 kB)
Requirement already satisfied: googleapis-common-protos~1.52 in /usr/local/lib/python3.11/dist-packages (from o
Requirement already satisfied: grpcio<2.0.0,>=1.63.2 in /usr/local/lib/python3.11/dist-packages (from openteleme
Collecting opentelemetry-exporter-otlp-proto-common==1.33.0 (from opentelemetry-exporter-otlp-proto-grpc<1.33.1,
Downloading opentelemetry_exporter_otlp_proto_common-1.33.0-py3-none-any.whl.metadata (1.9 kB)
Collecting opentelemetry-proto==1.33.0 (from opentelemetry-exporter-otlp-proto-grpc<1.33.1,>=1.18.0->qiskit_serv
Downloading opentelemetry_proto-1.33.0-py3-none-any.whl.metadata (2.4 kB)
Requirement already satisfied: protobuf<6.0,>=5.0 in /usr/local/lib/python3.11/dist-packages (from opentelemetry
Collecting opentelemetry-instrumentation==0.57b0 (from opentelemetry-instrumentation-requests>=0.40b0->qiskit_se
Downloading opentelemetry_instrumentation-0.57b0-py3-none-any.whl.metadata (6.7 kB)
Collecting opentelemetry-semantic-conventions==0.57b0 (from opentelemetry-instrumentation-requests>=0.40b0->qisk
Downloading opentelemetry_semantic_conventions-0.57b0-py3-none-any.whl.metadata (2.4 kB)
Collecting opentelemetry-util-http==0.57b0 (from opentelemetry-instrumentation-requests>=0.40b0->qiskit_serverle
Downloading opentelemetry_util_http-0.57b0-py3-none-any.whl.metadata (2.6 kB)
Requirement already satisfied: wrapt<2.0.0,>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from opentelemetr
INFO: pip is looking at multiple versions of opentelemetry-semantic-conventions to determine which version is co
Collecting opentelemetry-instrumentation-requests>=0.40b0 (from qiskit_serverless->qc-grader@ git+<https://github.com/qiskit-community/Quantum-Challenge>)
Downloading opentelemetry_instrumentation_requests-0.56b0-py3-none-any.whl.metadata (2.6 kB)
Collecting opentelemetry-instrumentation==0.56b0 (from opentelemetry-instrumentation-requests>=0.40b0->qiskit_se
Downloading opentelemetry_instrumentation-0.56b0-py3-none-any.whl.metadata (6.7 kB)
Collecting opentelemetry-semantic-conventions==0.56b0 (from opentelemetry-instrumentation-requests>=0.40b0->qisk
Downloading opentelemetry_semantic_conventions-0.56b0-py3-none-any.whl.metadata (2.4 kB)
Collecting opentelemetry-util-http==0.56b0 (from opentelemetry-instrumentation-requests>=0.40b0->qiskit_serverle
Downloading opentelemetry_util_http-0.56b0-py3-none-any.whl.metadata (2.6 kB)
Collecting opentelemetry-instrumentation-requests>=0.40b0 (from qiskit_serverless->qc-grader@ git+<https://github.com/qiskit-community/Quantum-Challenge>)
Downloading opentelemetry_instrumentation_requests-0.55b1-py3-none-any.whl.metadata (2.6 kB)
Collecting opentelemetry-instrumentation==0.55b1 (from opentelemetry-instrumentation-requests>=0.40b0->qiskit_se
Downloading opentelemetry_instrumentation-0.55b1-py3-none-any.whl.metadata (6.7 kB)
Collecting opentelemetry-semantic-conventions==0.55b1 (from opentelemetry-instrumentation-requests>=0.40b0->qisk
Downloading opentelemetry_semantic_conventions-0.55b1-py3-none-any.whl.metadata (2.5 kB)
Collecting opentelemetry-util-http==0.55b1 (from opentelemetry-instrumentation-requests>=0.40b0->qiskit_serverle
Downloading opentelemetry_util_http-0.55b1-py3-none-any.whl.metadata (2.6 kB)
Collecting opentelemetry-instrumentation-requests>=0.40b0 (from qiskit_serverless->qc-grader@ git+<https://github.com/qiskit-community/Quantum-Challenge>)
Downloading opentelemetry_instrumentation_requests-0.55b0-py3-none-any.whl.metadata (2.6 kB)
Collecting opentelemetry-instrumentation==0.55b0 (from opentelemetry-instrumentation-requests>=0.40b0->qiskit_se
Downloading opentelemetry_instrumentation-0.55b0-py3-none-any.whl.metadata (6.7 kB)
Collecting opentelemetry-semantic-conventions==0.55b0 (from opentelemetry-instrumentation-requests>=0.40b0->qisk
Downloading opentelemetry_semantic_conventions-0.55b0-py3-none-any.whl.metadata (2.5 kB)
Collecting opentelemetry-util-http==0.55b0 (from opentelemetry-instrumentation-requests>=0.40b0->qiskit_serverle
Downloading opentelemetry_util_http-0.55b0-py3-none-any.whl.metadata (2.6 kB)

```
Downloading opentelemetry_util_http-0.54b0-py3-none-any.whl.metadata (2.6 kB)
Collecting opentelemetry-instrumentation-requests>=0.40b0 (from qiskit_serverless->qc-grader@ git+https://github
  Downloading opentelemetry_instrumentation_requests-0.54b1-py3-none-any.whl.metadata (2.7 kB)
Collecting opentelemetry-instrumentation==0.54b1 (from opentelemetry-instrumentation-requests>=0.40b0->qiskit_se
  Downloading opentelemetry_instrumentation-0.54b1-py3-none-any.whl.metadata (6.8 kB)
Collecting opentelemetry-semantic-conventions==0.54b1 (from opentelemetry-instrumentation-requests>=0.40b0->qisk
  Downloading opentelemetry_semantic_conventions-0.54b1-py3-none-any.whl.metadata (2.5 kB)
Collecting opentelemetry-util-http==0.54b1 (from opentelemetry-instrumentation-requests>=0.40b0->qiskit_serverle
  Downloading opentelemetry_util_http-0.54b1-py3-none-any.whl.metadata (2.6 kB)
Collecting opentelemetry-instrumentation-requests>=0.40b0 (from qiskit_serverless->qc-grader@ git+https://github
  Downloading opentelemetry_instrumentation_requests-0.54b0-py3-none-any.whl.metadata (2.7 kB)
Collecting opentelemetry-instrumentation==0.54b0 (from opentelemetry-instrumentation-requests>=0.40b0->qiskit_se
  Downloading opentelemetry_instrumentation-0.54b0-py3-none-any.whl.metadata (6.8 kB)
Collecting opentelemetry-semantic-conventions==0.54b0 (from opentelemetry-instrumentation-requests>=0.40b0->qisk
  Downloading opentelemetry_semantic_conventions-0.54b0-py3-none-any.whl.metadata (2.5 kB)
Collecting opentelemetry-util-http==0.54b0 (from opentelemetry-instrumentation-requests>=0.40b0->qiskit_serverle
  Downloading opentelemetry_util_http-0.54b0-py3-none-any.whl.metadata (2.6 kB)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.11/dist-packages (from pydantic>
Requirement already satisfied: pydantic-core==2.33.2 in /usr/local/lib/python3.11/dist-packages (from pydantic>=
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/python3.11/dist-packages (from pydanti
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.0-
Collecting symengine<0.14,>=0.11 (from qiskit~=2.1.0->qiskit[visualization]~=2.1.0; extra == "qiskit"->qc-grader
  Downloading symengine-0.13.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (1.2 kB)
Requirement already satisfied: click>=7.0 in /usr/local/lib/python3.11/dist-packages (from ray<3,>=2.30->ray[def
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from ray<3,>=2.30->ray[defau
Requirement already satisfied: msgpack<2.0.0,>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from ray<3,>=2.
Requirement already satisfied: pyyaml in /usr/local/lib/python3.11/dist-packages (from ray<3,>=2.30->ray[default
Collecting aiohttp_cors (from ray[default]<3,>=2.30->qiskit_serverless->qc-grader@ git+https://github.com/qiskit
  Downloading aiohttp_cors-0.8.1-py3-none-any.whl.metadata (20 kB)
Collecting colorful (from ray[default]<3,>=2.30->qiskit_serverless->qc-grader@ git+https://github.com/qiskit-com
  Downloading colorful-0.5.7-py2.py3-none-any.whl.metadata (16 kB)
Collecting py-spy>=0.2.0 (from ray[default]<3,>=2.30->qiskit_serverless->qc-grader@ git+https://github.com/qiski
  Downloading py_spy-0.4.1-py2.py3-none-manylinux_2_5_x86_64.manylinux1_x86_64.whl.metadata (510 bytes)
Collecting opencensus (from ray[default]<3,>=2.30->qiskit_serverless->qc-grader@ git+https://github.com/qiskit-c
  Downloading opencensus-0.11.4-py2.py3-none-any.whl.metadata (12 kB)
Collecting opentelemetry-exporter-prometheus (from ray[default]<3,>=2.30->qiskit_serverless->qc-grader@ git+http
  Downloading opentelemetry_exporter_prometheus-0.57b0-py3-none-any.whl.metadata (1.8 kB)
Requirement already satisfied: smart_open in /usr/local/lib/python3.11/dist-packages (from ray[default]<3,>=2.30
Collecting virtualenv!=20.21.1,>=20.0.24 (from ray[default]<3,>=2.30->qiskit_serverless->qc-grader@ git+https://
  Downloading virtualenv-20.33.1-py3-none-any.whl.metadata (4.5 kB)
Requirement already satisfied: cryptography>=1.3 in /usr/local/lib/python3.11/dist-packages (from requests-ntlm>
Collecting pyspnego>=0.4.0 (from requests-ntlm>=1.1.0->qiskit-ibm-runtime->qc-grader@ git+https://github.com/qis
  Downloading pyspnego-0.11.2-py3-none-any.whl.metadata (5.4 kB)
Collecting aiobotocore<3.0.0,>=2.5.4 (from s3fs>=2023.6.0->qiskit_serverless->qc-grader@ git+https://github.com/
  Downloading aiobotocore-2.24.0-py3-none-any.whl.metadata (25 kB)
Collecting fsspec==2025.7.0 (from s3fs>=2023.6.0->qiskit_serverless->qc-grader@ git+https://github.com/qiskit-co
  Downloading fsspec-2025.7.0-py3-none-any.whl.metadata (12 kB)
Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.11/dist-packages (from seaborn>=0.9.0->qisk
Collecting pbr>=2.0.0 (from stevedore>=3.0.0->qiskit~=2.1.0->qiskit[visualization]~=2.1.0; extra == "qiskit"->qc
  Downloading pbr-6.1.1-py2.py3-none-any.whl.metadata (3.4 kB)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from sympy>=1.3->q
Collecting aioitertools<1.0.0,>=0.5.1 (from aiobotocore<3.0.0,>=2.5.4->s3fs>=2023.6.0->qiskit_serverless->qc-gra
  Downloading aioitertools-0.12.0-py3-none-any.whl.metadata (3.8 kB)
Collecting botocore<1.39.12,>=1.39.9 (from aiobotocore<3.0.0,>=2.5.4->s3fs>=2023.6.0->qiskit_serverless->qc-gra
  Downloading botocore-1.39.11-py3-none-any.whl.metadata (5.7 kB)
Collecting jmespath<2.0.0,>=0.7.1 (from aiobotocore<3.0.0,>=2.5.4->s3fs>=2023.6.0->qiskit_serverless->qc-grader@
  Downloading jmespath-1.0.1-py3-none-any.whl.metadata (7.6 kB)
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.11/dist-packages (from anyio->httpx>=0.25.
Requirement already satisfied: argon2-cffi-bindings in /usr/local/lib/python3.11/dist-packages (from argon2-cffi
Requirement already satisfied: cffi>=1.12 in /usr/local/lib/python3.11/dist-packages (from cryptography>=1.3->re
Requirement already satisfied: parso<0.9.0,>=0.8.4 in /usr/local/lib/python3.11/dist-packages (from jedi>=0.16->
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.11/dist-packages (
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.11/dist-packages (from jsonschema>=
Requirement already satisfied: rpds-py>=0.7.1 in /usr/local/lib/python3.11/dist-packages (from jsonschema>=4.18.
Collecting python-json-logger>=2.0.4 (from jupyter-events>=0.11.0->jupyter-server<3,>=2.4.0->jupyterlab->qc-gra
  Downloading python_json_logger-3.3.0-py3-none-any.whl.metadata (4.0 kB)
Collecting rfc3339-validator (from jupyter-events>=0.11.0->jupyter-server<3,>=2.4.0->jupyterlab->qc-grader@ git+
  Downloading rfc3339_validator-0.1.4-py2.py3-none-any.whl.metadata (1.5 kB)
Collecting rfc3986-validator>=0.1.1 (from jupyter-events>=0.11.0->jupyter-server<3,>=2.4.0->jupyterlab->qc-grade
  Downloading rfc3986_validator-0.1.1-py2.py3-none-any.whl.metadata (1.7 kB)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.11/dist-packages (from nbconvert>=6.4.4-
Requirement already satisfied: bleach!=5.0.0 in /usr/local/lib/python3.11/dist-packages (from bleach[css]!=5.0.0
Requirement already satisfied: defusedxml in /usr/local/lib/python3.11/dist-packages (from nbconvert>=6.4.4->jup
Requirement already satisfied: jupyterlab-pygments in /usr/local/lib/python3.11/dist-packages (from nbconvert>=6
Requirement already satisfied: mistune<4,>=2.0.3 in /usr/local/lib/python3.11/dist-packages (from nbconvert>=6.4
Requirement already satisfied: nbclient>=0.5.0 in /usr/local/lib/python3.11/dist-packages (from nbconvert>=6.4.4
Requirement already satisfied: pandocfilters>=1.4.1 in /usr/local/lib/python3.11/dist-packages (from nbconvert>=
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.11/dist-packages (from nbformat>=5
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.2->seaborn)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.2->seaborn)
```

Requirement already satisfied: ptyprocess>=0.5 in /usr/local/lib/python3.11/dist-packages (from pexpect>4.3->ipy
Requirement already satisfied: wcwidth in /usr/local/lib/python3.11/dist-packages (from prompt_toolkit<3.1.0,>=3
Collecting distlib<1,>=0.3.7 (from virtualenv!=20.21.1,>=20.0.24->ray[default]<3,>=2.30->qiskit_serverless->qc-g
 Downloading distlib-0.4.0-py2.py3-none-any.whl.metadata (5.2 kB)
Collecting opencensus-context>=0.1.3 (from opencensus->ray[default]<3,>=2.30->qiskit_serverless->qc-grader@ git+
 Downloading opencensus-context-0.1.3-py2.py3-none-any.whl.metadata (3.3 kB)
Requirement already satisfied: google-api-core<3.0.0,>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from op
INFO: pip is looking at multiple versions of opentelemetry-exporter-prometheus to determine which version is com
Collecting opentelemetry-exporter-prometheus (from ray[default]<3,>=2.30->qiskit_serverless->qc-grader@ git+<http>
 Downloading opentelemetry_exporter_prometheus-0.56b0-py3-none-any.whl.metadata (1.8 kB)
 Downloading opentelemetry_exporter_prometheus-0.55b1-py3-none-any.whl.metadata (1.9 kB)
 Downloading opentelemetry_exporter_prometheus-0.55b0-py3-none-any.whl.metadata (1.9 kB)
 Downloading opentelemetry_exporter_prometheus-0.54b1-py3-none-any.whl.metadata (1.9 kB)
 Downloading opentelemetry_exporter_prometheus-0.54b0-py3-none-any.whl.metadata (1.9 kB)
Collecting executing>=1.2.0 (from stack_data->ipython>=7.23.1->ipykernel->qc-grader@ git+<https://github.com/qisk>
 Downloading executing-2.2.0-py2.py3-none-any.whl.metadata (8.9 kB)
Collecting asttokens>=2.1.0 (from stack_data->ipython>=7.23.1->ipykernel->qc-grader@ git+<https://github.com/qisk>
 Downloading asttokens-3.0.0-py3-none-any.whl.metadata (4.7 kB)
Collecting pure-eval (from stack_data->ipython>=7.23.1->ipykernel->qc-grader@ git+<https://github.com/qiskit-comm>
 Downloading pure_eval-0.2.3-py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: webencodings in /usr/local/lib/python3.11/dist-packages (from bleach!=5.0.0->blea
Requirement already satisfied: tinycss2<1.5,>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from bleach[css]
Requirement already satisfied: pycparser in /usr/local/lib/python3.11/dist-packages (from cffi>=1.12->cryptograp
Requirement already satisfied: proto-plus<2.0.0,>=1.22.3 in /usr/local/lib/python3.11/dist-packages (from google
Requirement already satisfied: google-auth<3.0.0,>=2.14.1 in /usr/local/lib/python3.11/dist-packages (from googl
Collecting fqdn (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-server<3,>=2.4.0->jupyt
 Downloading fqdn-1.5.1-py3-none-any.whl.metadata (1.4 kB)
Collecting isoduration (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-server<3,>=2.4.0
 Downloading isoduration-20.11.0-py3-none-any.whl.metadata (5.7 kB)
Requirement already satisfied: jsonpointer>1.1.3 in /usr/local/lib/python3.11/dist-packages (from jsonschema[form
Collecting rfc3987-syntax>=1.1.0 (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-server
 Downloading rfc3987_syntax-1.1.0-py3-none-any.whl.metadata (7.7 kB)
Collecting uri-template (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-server<3,>=2.4.
 Downloading uri_template-1.3.0-py3-none-any.whl.metadata (8.8 kB)
Requirement already satisfied: webcolors>=24.6.0 in /usr/local/lib/python3.11/dist-packages (from jsonschema[for
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.11/dist-packages (from beautifulsoup4->nb
Requirement already satisfied: cachetools<6.0,>=2.0.0 in /usr/local/lib/python3.11/dist-packages (from google-au
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.11/dist-packages (from google-aut
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.11/dist-packages (from google-auth<3.0.0,
Collecting lark>=1.2.2 (from rfc3987-syntax>=1.1.0->jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.11.0->j
 Downloading lark-1.2.2-py3-none-any.whl.metadata (1.8 kB)
Collecting arrow>=0.15.0 (from isoduration->jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-s
 Downloading arrow-1.3.0-py3-none-any.whl.metadata (7.5 kB)
Collecting types-python-dateutil>=2.8.10 (from arrow>=0.15.0->isoduration->jsonschema[format-nongpl]>=4.18.0->ju
 Downloading types_python_dateutil-2.9.0.20250809-py3-none-any.whl.metadata (1.8 kB)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.11/dist-packages (from pyasn1-modu
Downloading ibm_platform_services-0.66.1-py3-none-any.whl (363 kB)
 363.9/363.9 kB 12.0 MB/s eta 0:00:00
Downloading jsonpickle-3.0.3-py3-none-any.whl (40 kB)
 40.8/40.8 kB 4.3 MB/s eta 0:00:00
Downloading networkx-3.2.1-py3-none-any.whl (1.6 MB)
 1.6/1.6 MB 33.6 MB/s eta 0:00:00
Downloading qiskit-2.1.1-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (7.5 MB)
 7.5/7.5 MB 65.5 MB/s eta 0:00:00
Downloading ipycytoscape-1.3.3-py2.py3-none-any.whl (3.6 MB)
 3.6/3.6 MB 67.0 MB/s eta 0:00:00
Downloading jupyterlab-4.4.5-py3-none-any.whl (12.3 MB)
 12.3/12.3 MB 75.4 MB/s eta 0:00:00
Downloading qiskit_aer-0.17.1-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (12.4 MB)
 12.4/12.4 MB 79.5 MB/s eta 0:00:00
Downloading qiskit_serverless-0.25.4-py3-none-any.whl (57 kB)
 57.0/57.0 kB 5.5 MB/s eta 0:00:00
Downloading certifi-2024.7.4-py3-none-any.whl (162 kB)
 163.0/163.0 kB 16.1 MB/s eta 0:00:00
Downloading cloudpickle-2.2.1-py3-none-any.whl (25 kB)
Downloading zipp-3.19.1-py3-none-any.whl (9.0 kB)
Downloading qiskit_ibm_runtime-0.40.1-py3-none-any.whl (3.2 MB)
 3.2/3.2 MB 20.9 MB/s eta 0:00:00
Downloading async_lru-2.0.5-py3-none-any.whl (6.1 kB)
Downloading ibm_cloud_sdk_core-3.24.1-py3-none-any.whl (75 kB)
 75.8/75.8 kB 8.0 MB/s eta 0:00:00
Downloading importlib_metadata-8.4.0-py3-none-any.whl (26 kB)
Downloading ipython-8.37.0-py3-none-any.whl (831 kB)
 831.9/831.9 kB 52.9 MB/s eta 0:00:00
Downloading ipywidgets-8.1.7-py3-none-any.whl (139 kB)
 139.8/139.8 kB 13.9 MB/s eta 0:00:00
Downloading jupyter_lsp-2.2.6-py3-none-any.whl (69 kB)
 69.4/69.4 kB 6.7 MB/s eta 0:00:00
Downloading jupyter_server-2.16.0-py3-none-any.whl (386 kB)
 386.9/386.9 kB 33.7 MB/s eta 0:00:00

```
Downloading jupyter_client-8.6.3-py3-none-any.whl (106 kB)
106.1/106.1 kB 12.0 MB/s eta 0:00:00
Downloading jupyterlab_server-2.27.3-py3-none-any.whl (59 kB)
59.7/59.7 kB 6.8 MB/s eta 0:00:00
Downloading opentelemetry_api-1.33.0-py3-none-any.whl (65 kB)
65.8/65.8 kB 7.4 MB/s eta 0:00:00
Downloading opentelemetry_exporter_otlp_proto_grpc-1.33.0-py3-none-any.whl (18 kB)
Downloading opentelemetry_exporter_otlp_proto_common-1.33.0-py3-none-any.whl (18 kB)
Downloading opentelemetry_proto-1.33.0-py3-none-any.whl (55 kB)
55.9/55.9 kB 5.4 MB/s eta 0:00:00
Downloading opentelemetry_instrumentation_requests-0.54b0-py3-none-any.whl (12 kB)
Downloading opentelemetry_instrumentation-0.54b0-py3-none-any.whl (31 kB)
Downloading opentelemetry_semantic_conventions-0.54b0-py3-none-any.whl (194 kB)
194.9/194.9 kB 15.0 MB/s eta 0:00:00
Downloading opentelemetry_util_http-0.54b0-py3-none-any.whl (7.3 kB)
Downloading opentelemetry_sdk-1.33.0-py3-none-any.whl (118 kB)
118.9/118.9 kB 11.8 MB/s eta 0:00:00
Downloading ray-2.48.0-cp311-cp311-manylinux2014_x86_64.whl (70.1 MB)
70.1/70.1 MB 9.2 MB/s eta 0:00:00
Downloading requests_ntlm-1.3.0-py3-none-any.whl (6.6 kB)
Downloading rustworkx-0.17.1-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.2 MB)
2.2/2.2 MB 70.4 MB/s eta 0:00:00
Downloading s3fs-2025.7.0-py3-none-any.whl (30 kB)
Downloading fsspec-2025.7.0-py3-none-any.whl (199 kB)
199.6/199.6 kB 19.3 MB/s eta 0:00:00
Downloading spectate-1.0.1-py2.py3-none-any.whl (11 kB)
Downloading stevedore-5.4.1-py3-none-any.whl (49 kB)
49.5/49.5 kB 5.3 MB/s eta 0:00:00
Downloading traitlets-5.14.3-py3-none-any.whl (85 kB)
85.4/85.4 kB 9.9 MB/s eta 0:00:00
Downloading aiobotocore-2.24.0-py3-none-any.whl (84 kB)
85.0/85.0 kB 8.9 MB/s eta 0:00:00
Downloading comm-0.2.3-py3-none-any.whl (7.3 kB)
Downloading Deprecated-1.2.18-py2.py3-none-any.whl (10.0 kB)
Downloading jedi-0.19.2-py2.py3-none-any.whl (1.6 MB)
1.6/1.6 MB 77.1 MB/s eta 0:00:00
Downloading json5-0.12.1-py3-none-any.whl (36 kB)
Downloading jupyter_events-0.12.0-py3-none-any.whl (19 kB)
Downloading jupyter_server_terminals-0.5.3-py3-none-any.whl (13 kB)
Downloading overrides-7.7.0-py3-none-any.whl (17 kB)
Downloading pbr-6.1.1-py2.py3-none-any.whl (108 kB)
109.0/109.0 kB 13.0 MB/s eta 0:00:00
Downloading py_spy-0.4.1-py2.py3-none-manylinux_2_5_x86_64.manylinux1_x86_64.whl (2.8 MB)
2.8/2.8 MB 99.1 MB/s eta 0:00:00
Downloading pyspnego-0.11.2-py3-none-any.whl (130 kB)
130.5/130.5 kB 15.3 MB/s eta 0:00:00
Downloading symengine-0.13.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (49.7 MB)
49.7/49.7 MB 15.8 MB/s eta 0:00:00
Downloading virtualenv-20.33.1-py3-none-any.whl (6.1 MB)
6.1/6.1 MB 44.0 MB/s eta 0:00:00
Downloading widgetsnbextension-4.0.14-py3-none-any.whl (2.2 MB)
2.2/2.2 MB 60.6 MB/s eta 0:00:00
Downloading aiohttp_cors-0.8.1-py3-none-any.whl (25 kB)
Downloading colorful-0.5.7-py2.py3-none-any.whl (201 kB)
201.5/201.5 kB 21.9 MB/s eta 0:00:00
Downloading opencensus-0.11.4-py2.py3-none-any.whl (128 kB)
128.2/128.2 kB 15.2 MB/s eta 0:00:00
Downloading opentelemetry_exporter_prometheus-0.54b0-py3-none-any.whl (12 kB)
Downloading stack_data-0.6.3-py3-none-any.whl (24 kB)
Downloading aioitertools-0.12.0-py3-none-any.whl (24 kB)
Downloading asttokens-3.0.0-py3-none-any.whl (26 kB)
Downloading botocore-1.39.11-py3-none-any.whl (13.9 MB)
13.9/13.9 MB 63.4 MB/s eta 0:00:00
Downloading distlib-0.4.0-py2.py3-none-any.whl (469 kB)
469.0/469.0 kB 37.4 MB/s eta 0:00:00
Downloading executing-2.2.0-py2.py3-none-any.whl (26 kB)
Downloading jmespath-1.0.1-py3-none-any.whl (20 kB)
Downloading opencensus_context-0.1.3-py2.py3-none-any.whl (5.1 kB)
Downloading python_json_logger-3.3.0-py3-none-any.whl (15 kB)
Downloading rfc3986_validator-0.1.1-py2.py3-none-any.whl (4.2 kB)
Downloading pure_eval-0.2.3-py3-none-any.whl (11 kB)
Downloading rfc3339_validator-0.1.4-py2.py3-none-any.whl (3.5 kB)
Downloading rfc3987_syntax-1.1.0-py3-none-any.whl (8.0 kB)
Downloading fqdn-1.5.1-py3-none-any.whl (9.1 kB)
Downloading isoduration-20.11.0-py3-none-any.whl (11 kB)
Downloading uri_template-1.3.0-py3-none-any.whl (11 kB)
Downloading arrow-1.3.0-py3-none-any.whl (66 kB)
66.4/66.4 kB 7.3 MB/s eta 0:00:00
Downloading lark-1.2.2-py3-none-any.whl (111 kB)
111.0/111.0 kB 12.6 MB/s eta 0:00:00
```

```

Downloaded types_python_dateutil-2.9.0.20250809-py3-none-any.whl (17 kB)
Building wheels for collected packages: qc-grader, pylatexenc
  Building wheel for qc-grader (setup.py) ... done
  Created wheel for qc-grader: filename=qc_grader-0.22.12-py3-none-any.whl size=2519069 sha256=cb2b077a68114d781
  Stored in directory: /tmp/pip-ephem-wheel-cache-hiowihj8/wheels/56/7f/8b/c045ba8136342d5e0361484d7e6e0d24f9b78
  Building wheel for pylatexenc (setup.py) ... done
  Created wheel for pylatexenc: filename=pylatexenc-2.10-py3-none-any.whl size=136817 sha256=3e5326c58af91db95ce
  Stored in directory: /root/.cache/pip/wheels/b1/7a/33/9fdd892f784ed4afda62b685ae3703adf4c91aa0f524c28f03
Successfully built qc-grader pylatexenc
Installing collected packages: pylatexenc, py-spy, pure-eval, opencensus-context, distlib, colorful, zipp, widge
  Attempting uninstall: zipp
    Found existing installation: zipp 3.23.0
    Uninstalling zipp-3.23.0:
      Successfully uninstalled zipp-3.23.0
  Attempting uninstall: widgetsnbextension
    Found existing installation: widgetsnbextension 3.6.10
    Uninstalling widgetsnbextension-3.6.10:
      Successfully uninstalled widgetsnbextension-3.6.10
  Attempting uninstall: traitlets
    Found existing installation: traitlets 5.7.1
    Uninstalling traitlets-5.7.1:
      Successfully uninstalled traitlets-5.7.1
  Attempting uninstall: networkx
    Found existing installation: networkx 3.5
    Uninstalling networkx-3.5:
      Successfully uninstalled networkx-3.5
  Attempting uninstall: jsonpickle
    Found existing installation: jsonpickle 4.1.1
    Uninstalling jsonpickle-4.1.1:
      Successfully uninstalled jsonpickle-4.1.1
  Attempting uninstall: fsspec
    Found existing installation: fsspec 2025.3.0
    Uninstalling fsspec-2025.3.0:
      Successfully uninstalled fsspec-2025.3.0
  Attempting uninstall: cloudpickle
    Found existing installation: cloudpickle 3.1.1
    Uninstalling cloudpickle-3.1.1:
      Successfully uninstalled cloudpickle-3.1.1
  Attempting uninstall: certifi
    Found existing installation: certifi 2025.8.3
    Uninstalling certifi-2025.8.3:
      Successfully uninstalled certifi-2025.8.3
  Attempting uninstall: importlib-metadata
    Found existing installation: importlib_metadata 8.7.0
    Uninstalling importlib_metadata-8.7.0:
      Successfully uninstalled importlib_metadata-8.7.0
  Attempting uninstall: jupyter-client
    Found existing installation: jupyter-client 6.1.12
    Uninstalling jupyter-client-6.1.12:
      Successfully uninstalled jupyter-client-6.1.12
  Attempting uninstall: ipython
    Found existing installation: ipython 7.34.0
    Uninstalling ipython-7.34.0:
      Successfully uninstalled ipython-7.34.0
  Attempting uninstall: ipywidgets
    Found existing installation: ipywidgets 7.7.1
    Uninstalling ipywidgets-7.7.1:
      Successfully uninstalled ipywidgets-7.7.1
  Attempting uninstall: jupyter-server
    Found existing installation: jupyter-server 1.16.0
    Uninstalling jupyter-server-1.16.0:
      Successfully uninstalled jupyter-server-1.16.0
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This
google-colab 1.0.0 requires ipython==7.34.0, but you have ipython 8.37.0 which is incompatible.
google-colab 1.0.0 requires jupyter-server==1.16.0, but you have jupyter-server 2.16.0 which is incompatible.
jupyter-kernel-gateway 2.5.2 requires jupyter-client<8.0,>=5.2.0, but you have jupyter-client 8.6.3 which is inc
torch 2.6.0+cu124 requires nvidia-cublas-cu12==12.4.5.8; platform_system == "Linux" and platform_machine == "x86
torch 2.6.0+cu124 requires nvidia-cuda-cupti-cu12==12.4.127; platform_system == "Linux" and platform_machine ==
torch 2.6.0+cu124 requires nvidia-cuda-nvrtc-cu12==12.4.127; platform_system == "Linux" and platform_machine ==
torch 2.6.0+cu124 requires nvidia-cuda-runtime-cu12==12.4.127; platform_system == "Linux" and platform_machine =
torch 2.6.0+cu124 requires nvidia-cudnn-cu12==9.1.0.70; platform_system == "Linux" and platform_machine == "x86_
torch 2.6.0+cu124 requires nvidia-cufft-cu12==11.2.1.3; platform_system == "Linux" and platform_machine == "x86_
torch 2.6.0+cu124 requires nvidia-curand-cu12==10.3.5.147; platform_system == "Linux" and platform_machine == "x
torch 2.6.0+cu124 requires nvidia-cusolver-cu12==11.6.1.9; platform_system == "Linux" and platform_machine == "x
torch 2.6.0+cu124 requires nvidia-cusparse-cu12==12.3.1.170; platform_system == "Linux" and platform_machine ==
torch 2.6.0+cu124 requires nvidia-nccl-cu12==2.21.5; platform_system == "Linux" and platform_machine == "x86_64"
torch 2.6.0+cu124 requires nvidia-nvjitlink-cu12==12.4.127; platform_system == "Linux" and platform_machine == "
datasets 4.0.0 requires fsspec[http]<=2025.3.0,>=2023.1.0, but you have fsspec 2025.7.0 which is incompatible.
notebook 6.5.7 requires jupyter-client<8,>=5.3.4, but you have jupyter-client 8.6.3 which is incompatible.

```

```
distributed 2025.5.0 requires cloudpickle>=3.0.0, but you have cloudpickle 2.2.1 which is incompatible.
gcsfs 2025.3.0 requires fsspec==2025.3.0, but you have fsspec 2025.7.0 which is incompatible.
dask 2025.5.0 requires cloudpickle>=3.0.0, but you have cloudpickle 2.2.1 which is incompatible.
Successfully installed aiobotocore-2.24.0 aiohttp_cors-0.8.1 aioitertools-0.12.0 arrow-1.3.0 asttokens-3.0.0 asy
WARNING: Upgrading ipython, ipykernel, tornado, prompt-toolkit, pyzmq can cause your runtime to repeatedly crash or behave in unexpected ways and is not recommended. If your runtime won't connect or execute code, you can reset it with "Disconnect and delete runtime" from the "Runtime" menu.
```

WARNING: The following packages were previously imported in this runtime:
[IPython,certifi,traitlets]
You must restart the runtime in order to use newly installed versions.

RESTART SESSION

```
import qiskit
import qc_grader
print(f"Qiskit version: {qiskit.__version__}")
print(f"Grader version: {qc_grader.__version__}")
```

```
⇒ Qiskit version: 2.1.1
Grader version: 0.22.12
```

You should have Qiskit version $\geq 2.0.0$ and Grader $\geq 0.22.9$. If you see a lower version, you need to restart your kernel and reinstall the grader. Also make sure you have set up everything according to lab 0 and test it with the cell below.

Save your API key to track your progress and have access to the quantum computers

```
#your_api_key = "deleteThisAndPasteYourAPIKeyHere"
#your_crn = "deleteThisAndPasteYourCRNHere"
```

```
from qiskit_ibm_runtime import QiskitRuntimeService
```

```
QiskitRuntimeService.save_account(
    channel="ibm_quantum_platform",
    token=your_api_key,
    instance=your_crn,
    name="qgss-2025",
    overwrite=True
)
```

```
# Check that the account has been saved properly
from qiskit_ibm_runtime import QiskitRuntimeService
```

```
service = QiskitRuntimeService(name="qgss-2025")
service.saved_accounts()
```

Imports

The cell below imports these necessary libraries and prints your Qiskit version.

```
# Essential libraries
import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display
from PIL import Image
import io

from qiskit import QuantumCircuit
from qiskit.circuit import Parameter
from qiskit.visualization import plot_histogram, plot_distribution
from qiskit_ibm_runtime import Options, Session, SamplerV2 as Sampler
from qiskit.result import marginal_distribution

from qiskit.transpiler import generate_preset_pass_manager
from qiskit_ibm_runtime import QiskitRuntimeService
from qiskit_aer import AerSimulator

from qc_grader.challenges.qgss_2025 import (
    grade_lab1_ex1_1,
    grade_lab1_ex1_2,
    grade_lab1_ex1_3,
    grade_lab1_ex1_4,
    grade_lab1_ex2,
    grade_lab1_ex3,
    grade_lab1_ex4,
    grade_lab1_ex5,
    grade_lab1_ex6
)
```

Table of Contents

1. Superposition, Interference and Measurement

1. Double-slit Experiment - Superposition, Interference

2. Schrödinger’s Cat and Double-slit Revisit

2. Entanglement

1. The Curious Case of Alice, Bob, and the Unbeatable Game - CHSH Game

2. Quantum Teleportation - Sending Secrets with Spooky Action

1. Teleportation on a Simulator

3. (Bonus Challenge) Teleportation on Real Quantum Hardware

Welcome to the very first QGSS Lab of 2025, the International Year of Quantum! In the first lab, we'll explore four of the most important concepts of the quantum world: superposition, interference, measurement, and entanglement - through hands-on experiments using Qiskit.

✓ Chapter 1: Superposition, Interference and Measurement

To kick off this exciting journey, we'll begin by diving into superposition, interference, and quantum measurement through the famous double-slit experiment. **Superposition** means a quantum particle - for example, an electron or a photon — can exist in multiple states at once, like being in two places or having two energies at the same time. It's like flipping a coin with the result of heads and tails at the same time... until you look! **Interference** describes what happens when these quantum possibilities, behaving like waves, overlap. If they are in phase, they reinforce each other (constructive interference), and if out of phase, they can weaken or cancel each other out (destructive interference).

One of the most famous experiments demonstrating these concepts is the **double-slit experiment**, first done by Thomas Young in 1801 and later repeated with individual particles like electrons. When no one watches which slit the particle goes through, the result is a beautiful interference pattern, like waves overlapping. But the moment you **measure** which slit it goes through, the pattern disappears. It's as if the particle knows it's being watched!

This strange idea was taken to the next level by Erwin Schrödinger in 1935 with the famous thought experiment known as **Schrödinger's Cat**, where a cat can be both alive and dead until someone opens the box to check. Measurement in quantum mechanics doesn't just observe reality, it shapes it.

Ready to experience quantum weirdness yourself? With Qiskit, you can do the double-slit experiment - with and without measurement - and see for yourself the beautiful patterns of superposition and interference, as well as the effects measurement has on quantum systems. It's a fun and eye-opening way to explore the mysteries of the quantum world!

✓ Through the Slits: Where Quantum Weirdness Begins

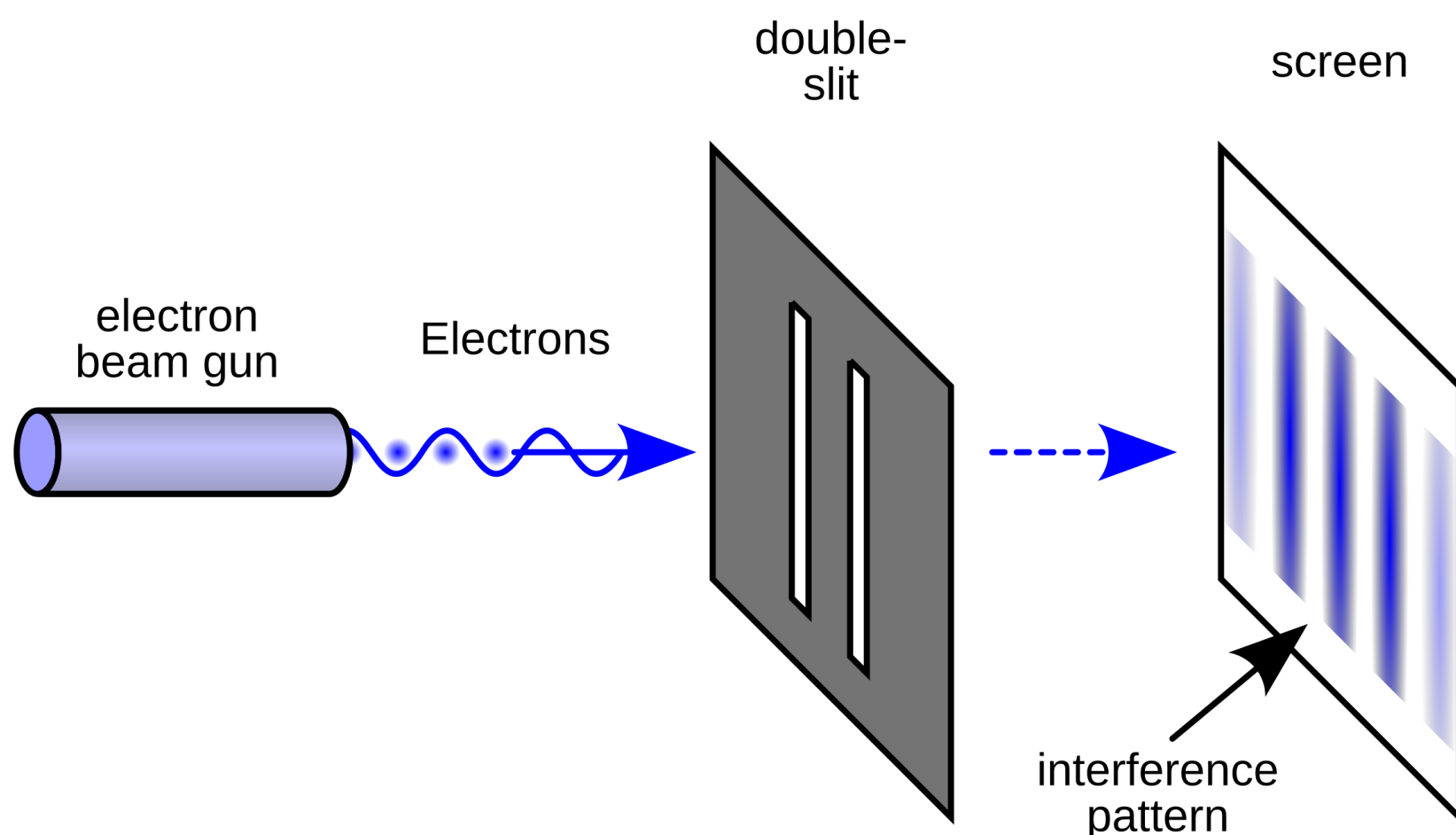


image from wikipedia

In his paper "Experiments and calculations relative to physical optics" 1803, Thomas Young explains about his experiments in this quotation:

I made a small hole in a window-shutter, and covered it with a piece of thick paper, which I perforated with a fine needle. For greater convenience of observation, I placed a small looking glass without the window-shutter, in such a position as to reflect the sun's light, in a direction nearly horizontal, upon the opposite wall, and to cause the cone of diverging light to pass over a table, on which were several little screens of card-paper. [1](#)

With this experiment, he could observe beautiful fringe patterns by light and explained this comes from the differences in the lengths of the paths of two rays. George Paget Thomson later reproduced a similar experiment using electrons (electron diffraction experiment), and showed that even particles like electrons exhibit wave-like behavior, as demonstrated in his 1927 paper. [2](#) Later in 1965, Feynman explained the double-slit experiment in his "Lectures on Physics, Vol. 3, Chapter 1", and explained this comes from the superposition and interference concepts of quantum mechanics.

Now we can reproduce this fascinating quantum phenomenon using Qiskit, without complex physical equipment. Let's see how superposition and interference are implemented in quantum circuits.

The first step is mapping the physical double-slit experiment to a quantum circuit, which is your first exercise.

Exercise 1: Build a quantum circuit for the double-slit experiment

- Exercise 1-1: Make a slit

Your Goal: Create a quantum circuit that models a particle (qubit) passing through two slits, achieving a superposition of being in the $|0\rangle$ (lower slit) and $|1\rangle$ (upper slit) states.

Let's define the upper slit as the $|1\rangle$ state of a qubit and the lower slit as the $|0\rangle$ state. Implement a quantum circuit where a qubit, initially in $|0\rangle$, passes through the slits in a superposition of $|0\rangle$ and $|1\rangle$ using the [Hadamard gate](#). Draw your circuit.

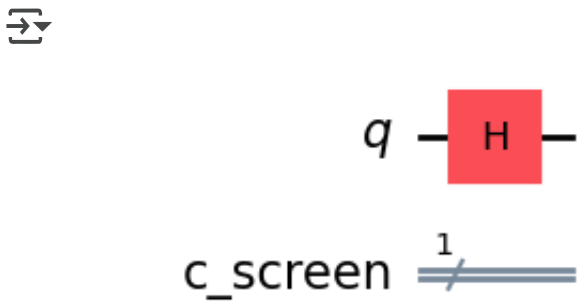
Hint: It can be beneficial for clarity to assign specific names to QuantumRegister (e.g., name='q') and ClassicalRegister (e.g., name='c_screen') when interpreting measurement data.

```
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister

qr = QuantumRegister(1, name='q')
cr = ClassicalRegister(1, name='c_screen')

double_slit = QuantumCircuit(qr, cr)
# your code here
double_slit.h(0) #apply first H gate to create superposition

# end of your code
double_slit.draw('mpl')
```



✓ Reference for personal use

Qiskit - Double-Slit Experiment on a Quantum Computer: Wave Particle Duality Explained | Paradoxes Ep. 05:
<https://www.youtube.com/watch?v=CI41yWg38Oo>

```
# Submit your answer using the following code
grade_lab1_ex1_1(double_slit)
```



Could not find a valid API server or the API servers are down right now.
Failed: None
Unable to grade your answer.

Exercise 1: Build a quantum circuit for the double-slit experiment

- Exercise 1-2: Make a screen

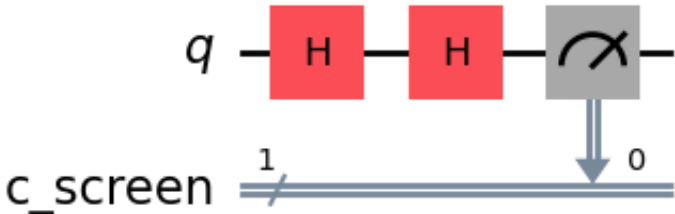
Your Goal: Extend the previous circuit to model the screen where interference occurs. Specifically, implement the center of the screen where no phase difference is introduced, and then measure the qubit.

Now, implement the screen where the two superposed quantum states create an interference pattern by adding a gate. First, let's implement the center of the screen, where the two beams combine with no phase difference (resulting in the $|0\rangle$ state). Then measure the qubit from `qr` and store the result in `c_screen`.

Hint: You can use the Hadamard gate again.

```
# your code here
double_slit.h(0) #apply second H gate to combine paths
double_slit.measure(qr, cr) #measure(qubit, cbit)
```

```
# end of your code
double_slit.draw('mpl')
```



```
# Submit your answer using the following code
grade_lab1_ex1_2(double_slit)
```



Could not find a valid API server or the API servers are down right now.
Failed: None
Unable to grade your answer.

Let's check the execution result. Assuming the screen detects the qubit in the $|0\rangle$ state, the following code runs the circuit on an ideal simulator. A probability of 1 means 100%.

Note on Retrieving Measurement Results with SamplerV2

When using *SamplerV2* in Qiskit, how you retrieve measurement counts depends on how you've set up your classical registers and measurements:

1. **Named Classical Register in *QuantumCircuit*:** If you define your circuit with a named classical register, e.g., *cr = ClassicalRegister(1, name='my_results')* and *qc = QuantumCircuit(qr, cr)*, and then measure to it *qc.measure(qr[0], cr[0])*, you access the counts using that name:

```
```python
result = job.result()
counts = result[0].data.my_results.get_counts()
```
```

In our double-slit circuit, we used **cr = ClassicalRegister(1, name='c_screen')**, so we will use **result[0].data.c*

3. **measure_all():** If you use *qc.measure_all()*, Qiskit automatically adds classical bits and names the output data field *meas* :

```
```python
qc.measure_all()
...
counts = result[0].data.meas.get_counts()
```
```

5. **Implicit Classical Bits (No Named Register in *QuantumCircuit*):** If you create a circuit like *qc = QuantumCircuit(1, 1)* (1 qubit, 1 classical bit) or *qc = QuantumCircuit(QuantumRegister(1), ClassicalRegister(1))* without naming the classical register in its constructor, and then use *qc.measure(0, 0)*, the *SamplerV2* might store results under a default name (often *c*, or based on the classical bit indices like *c0*). For *qc = QuantumCircuit(1,1)* and *qc.measure(0,0)*, the output might be accessed as follows:

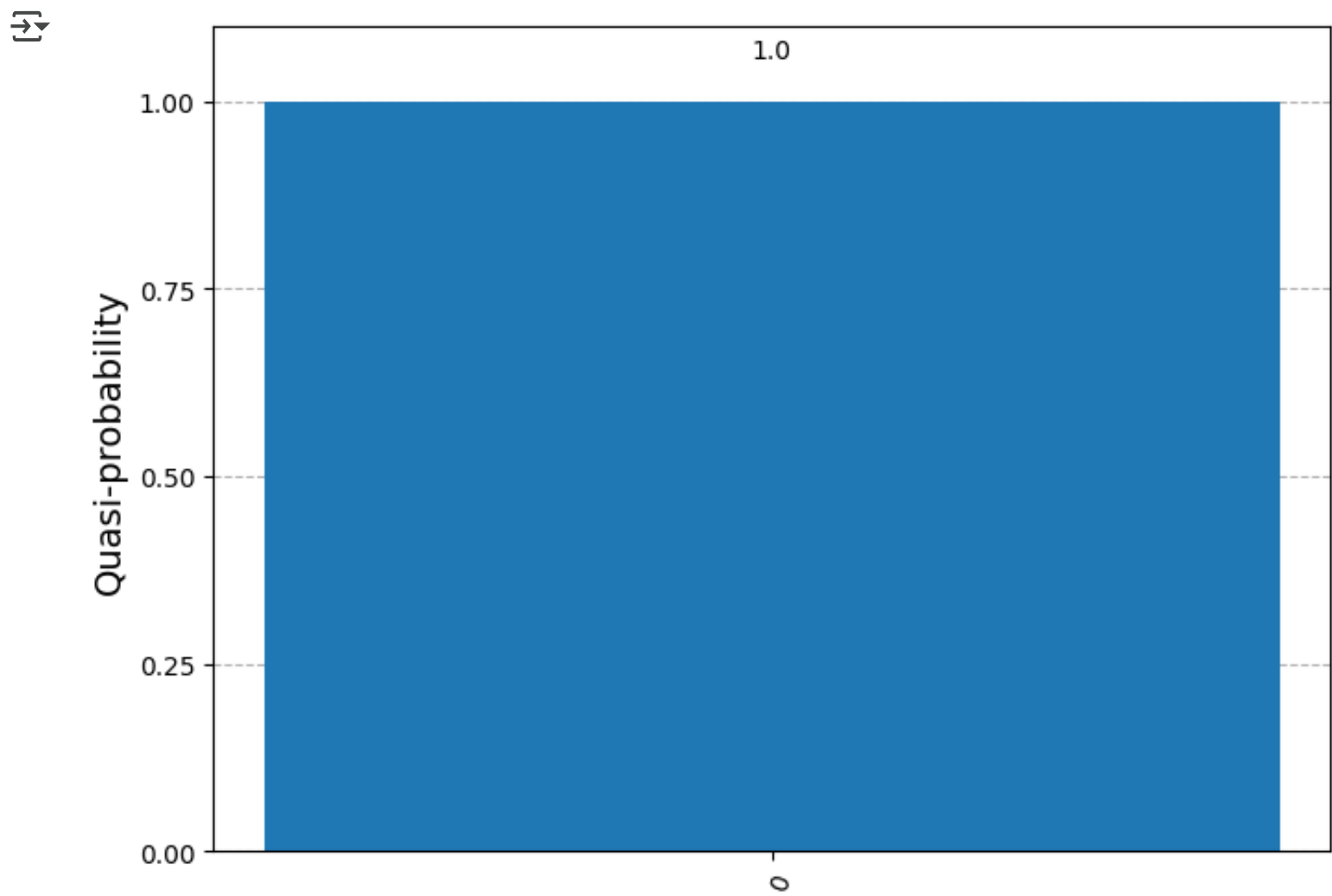
```
# counts = result[0].data.c0.get_counts() # If single bit named c0, the indice can vary. You can see the acti
```

```
# use simulator
backend = AerSimulator()

# make quantum circuit compatible to the backend
pm = generate_preset_pass_manager(backend = backend, optimization_level=3)
qc_isa = pm.run(double_slit)

# run and get counts
sampler = Sampler(mode=backend)
counts = sampler.run([qc_isa], shots = 1000).result()[0].data.c_screen.get_counts()

plot_distribution(counts)
```



Exercise 1: Build a quantum circuit for the double-slit experiment

- Exercise 1-3: Make a difference

Your Goal: Modify the double-slit circuit to introduce a phase difference between the two paths (slits) and observe its effect on the measurement outcome at the screen.

Now, let's implement another part of the screen. The path difference between the two beams is implemented as a phase difference between $|0\rangle$ and $|1\rangle$. Apply a phase of $\pi/2$ only to the $|1\rangle$ state of the superposed quantum state and observe the measurement results. (Hint: Use a gate that applies a phase to the $|1\rangle$ state. The [P_gate](#) and the [S_gate](#) are representative examples.)

```
import math

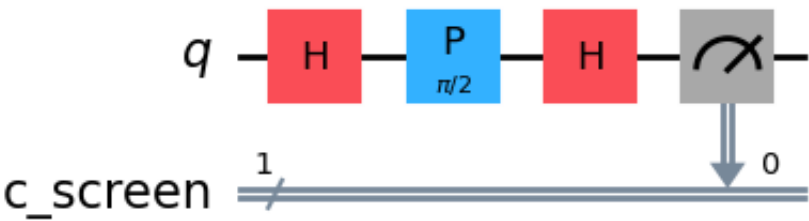
qr = QuantumRegister(1, name='q')
cr = ClassicalRegister(1, name='c_screen')

double_slit_with_difference = QuantumCircuit(qr, cr)
double_slit_with_difference.h(0)

#your code here
double_slit_with_difference.p(math.pi/2, qr)  #p gate applies phase change only when state is 1, remains unchange i

#end of your code

double_slit_with_difference.h(0)
double_slit_with_difference.measure(qr, cr)
double_slit_with_difference.draw('mpl')
```

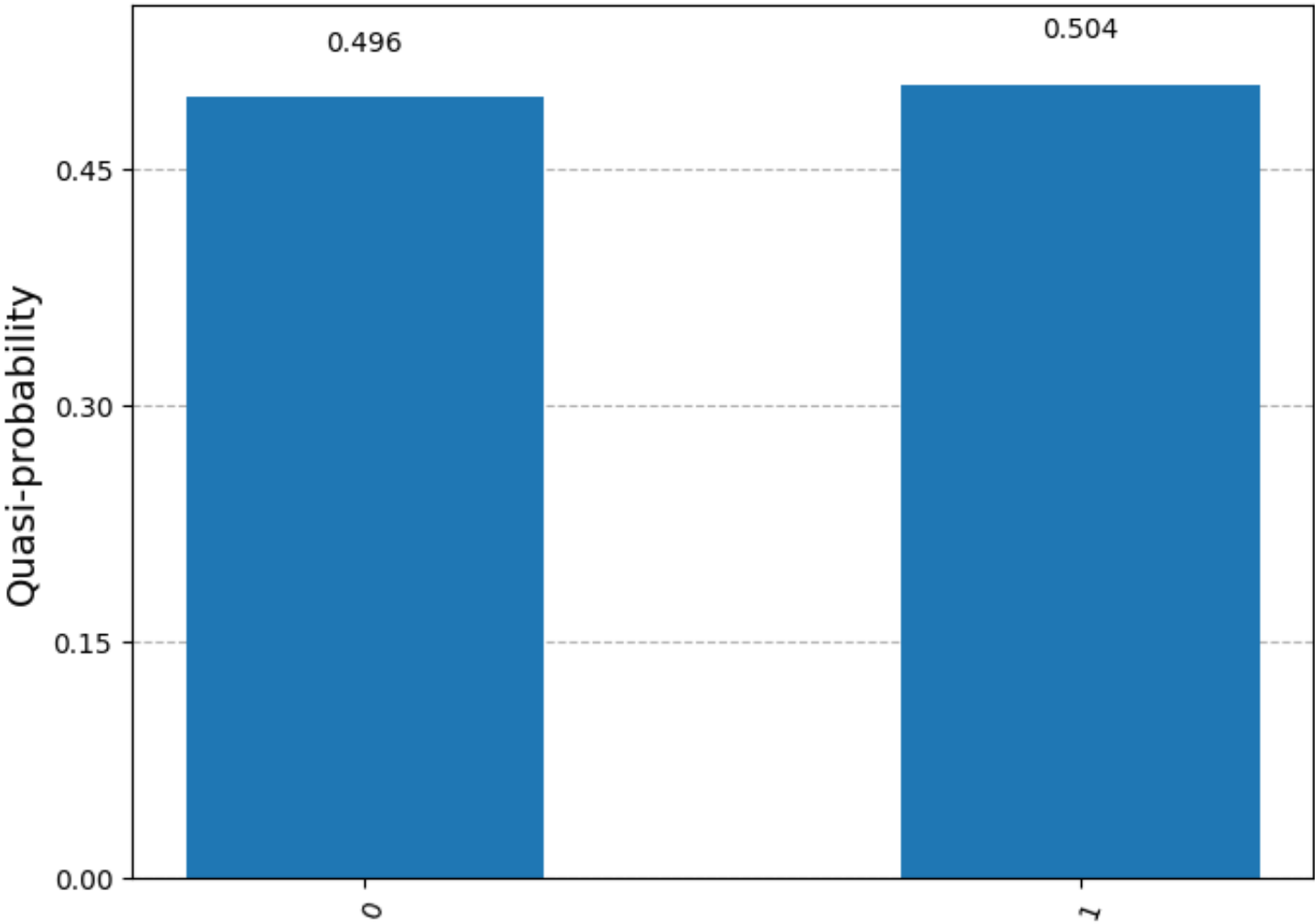


```
# Submit your answer using the following code
grade_lab1_ex1_3(double_slit_with_difference)
```

 Could not find a valid API server or the API servers are down right now.
Failed: None
Unable to grade your answer.

```
qc_isa = pm.run(double_slit_with_difference)

#run and get counts
sampler = Sampler(mode=backend)
counts = sampler.run([qc_isa], shots=10000).result()[0].data.c_screen.get_counts()
plot_distribution(counts)
```



As you can see, the probability of measuring $|0\rangle$ decreased to approximately 0.5 (50%), signifying reduced brightness at this point on the screen.

Before we go further let's do a simple mathematics demonstration of this process in regards to superposition, interference, and the probability of measuring a quantum state. You can skip this part if you are already familiar with this concept.

▸ Superposition, Interference and the Probability of Measurement

Let's conclude Exercise 1 by generating the fringe pattern using a range of ϕ .

Exercise 1: Build a quantum circuit for the double-slit experiment

- Exercise 1-4: Beautiful Fringes

Your Goal: Create a parameterized quantum circuit where the phase difference ϕ can be varied, allowing you to simulate and visualize the complete interference fringe pattern.

Qiskit allows [using Parameters in quantum circuits](#). We'll use this with the `Sampler` to observe interference fringes.

Define a parameterized quantum circuit for the double-slit experiment with a variable parameter ϕ . A common structure is H-gate, then $P(\phi)$, then another H-gate, before measurement. Measure `q` and save to `c_screen`.

```
phi = Parameter('phi')

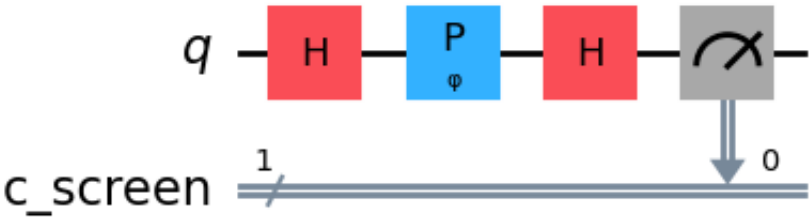
qr = QuantumRegister(1, name='q')
cr = ClassicalRegister(1, name='c_screen')

double_slit_fringe = QuantumCircuit(qr, cr)

#your code here
double_slit_fringe.h(0)
double_slit_fringe.p(phi, qr)
double_slit_fringe.h(0)
double_slit_fringe.measure(qr, cr)

#end of your code

double_slit_fringe.draw('mpl')
```



```
# Submit your answer using the following code
grade_lab1_ex1_4(double_slit_fringe)
```

Could not find a valid API server or the API servers are down right now.
Failed: None
Unable to grade your answer.

Excellent! Now, let's use this parameterized circuit to plot the fringe pattern using matplotlib's heat map. The code below generates 100 phase values, runs the circuits (1000 shots each) via `Sampler`, stores the probability of measuring $|0\rangle$, and plots the heat map.

Resource limit

When running the code below on an actual backend, increasing the number of parameters or shots consumes more QPU time than might be expected. The current configuration (100 parameterized circuits + 1000 shots) uses less than 1 minute of QPU time, so please try to maintain these settings if possible.

```

φ_lst = np.linspace(-3*np.pi, 3*np.pi, 100)
qc_isa = pm.run(double_slit_fringe)

φ_hit = []
dist = sampler.run([(qc_isa, φ_lst)], shots=1000).result()[0].data.c_screen

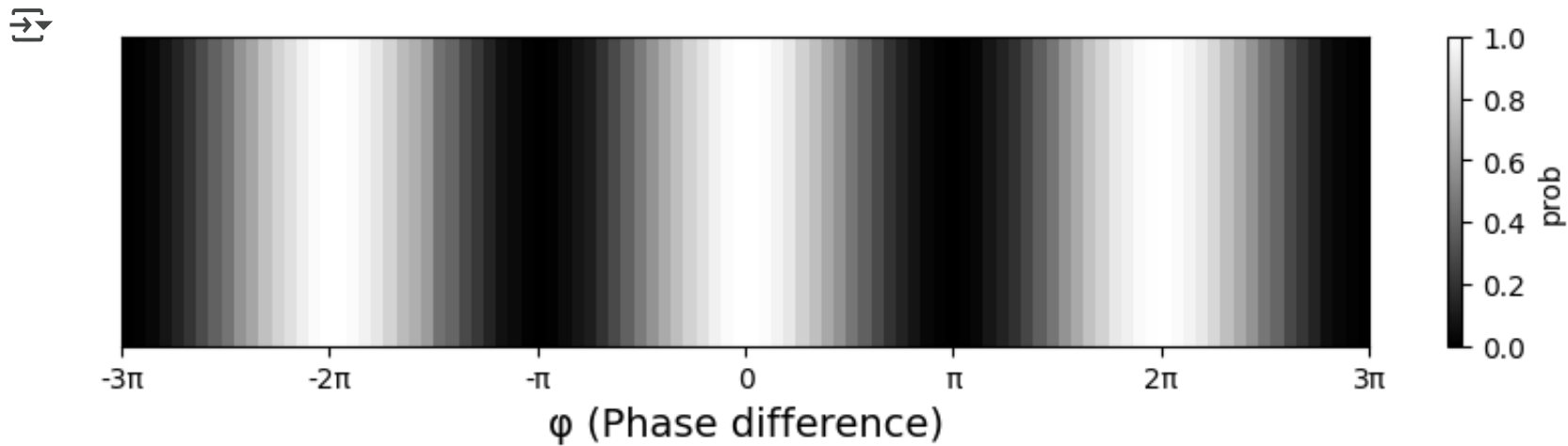
for i in range(len(φ_lst)):
    result = dist[i].get_counts()
    if '0' in result:
        φ_hit.append(result['0']/1000)
    else:
        φ_hit.append(0)

#plot heat map
φ_hit_2d = np.array(φ_hit).reshape(1, -1)

plt.figure(figsize=(10, 2))
plt.imshow(φ_hit_2d, cmap='gray', aspect='auto', extent=[-3*np.pi, 3*np.pi, 0, 0.1])

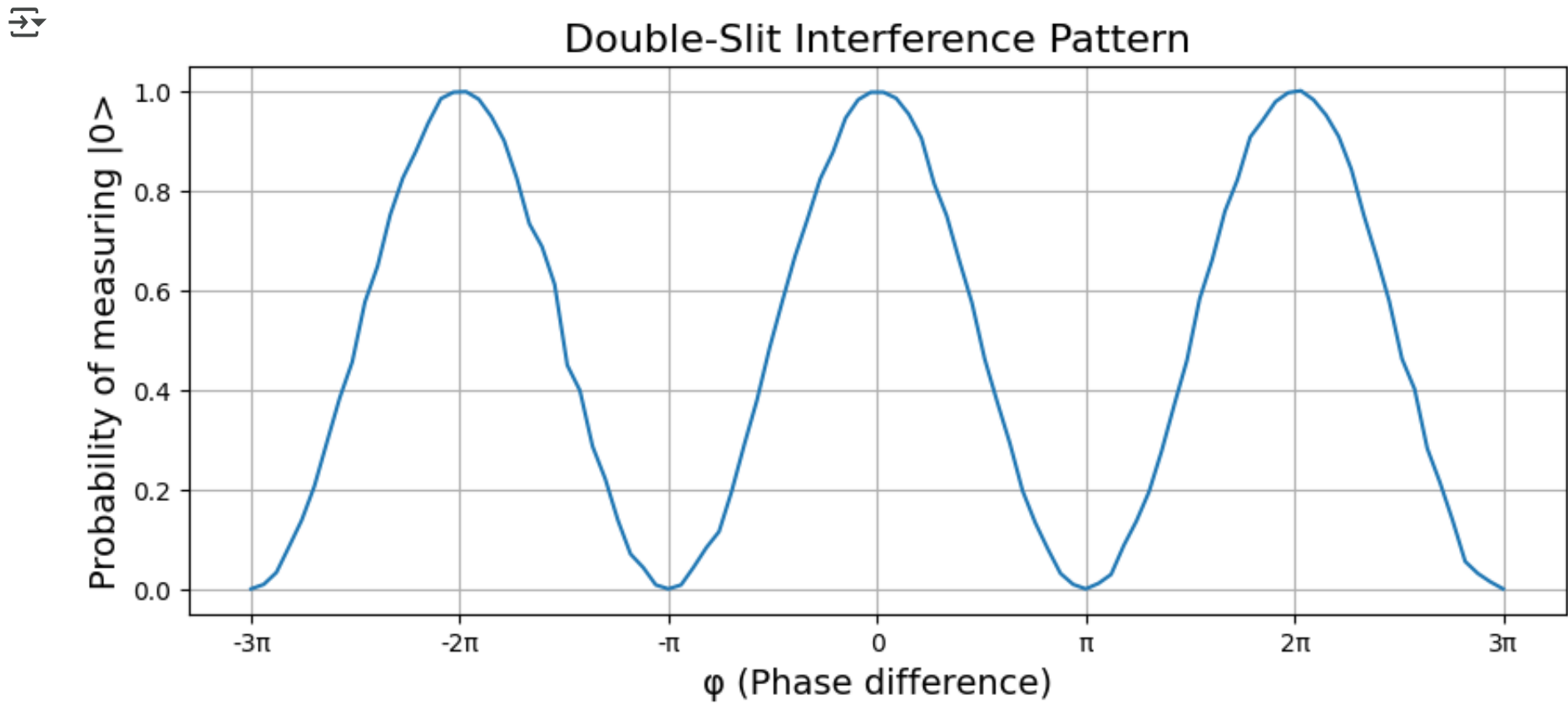
plt.xlabel('φ (Phase difference)', fontsize=14)
plt.colorbar(label='prob')
plt.xticks(ticks=[-3*np.pi, -2*np.pi, -np.pi, 0, np.pi, 2*np.pi, 3*np.pi],
           labels=['-3π', '-2π', '-π', '0', 'π', '2π', '3π'])
plt.yticks([])
plt.show()

```



Fantastic! You've reproduced the double-slit experiment's fringes using qubits. Next, we'll explore quantum measurement with Schrödinger's cat and then see how observation affects the double-slit outcome.

```
plt.figure(figsize=(10, 4))
plt.plot(phi_lst, phi_hit)
plt.xlabel('phi (Phase difference)', fontsize=14)
plt.ylabel('Probability of measuring |0>', fontsize=14)
plt.title('Double-Slit Interference Pattern', fontsize=16)
plt.xticks(ticks=[-3 * np.pi, -2 * np.pi, -np.pi, 0, np.pi, 2 * np.pi, 3 * np.pi],
           labels=['-3pi', '-2pi', '-pi', '0', 'pi', '2pi', '3pi'])
plt.grid(True)
plt.show()
```



✓ Schrödinger's Cat: Exploring Quantum Superposition and the Act of Observation

This thought experiment by Erwin Schrödinger (1935, "[Die gegenwärtige Situation in der Quantenmechanik](#)") illustrates quantum superposition and measurement's role in collapsing possibilities into a single outcome. A cat is sealed in a box with a radioactive atom. If the atom is in a superposition of decayed/not decayed, the cat, entangled with it, should be in a superposition of alive/dead until observed.

Here, we'll use a rotational gate. Imagine a cat disliking the $|1\rangle$ state. Let's see if the cat is happy or upset when we "open the box" (measure the qubit). Be careful! An angry quantum cat might scratch you.

Exercise 2: Is the cat happy or grumpy?

Your Goal: Create a quantum circuit that prepares a qubit in a superposition state using an $R_X(\theta)$ gate, then simulate a single measurement to determine if the "cat" (represented by the qubit state) is "happy" ($|0\rangle$) or "grumpy" ($|1\rangle$).

Complete the Python code below using the [Rotational X gate](#) with $\theta \in [0, 2\pi]$ to prepare a qubit in various superposition states. Then, measure once to see if the cat is happy ($|0\rangle$) or grumpy ($|1\rangle$). θ will be given to you by the slider.

```
def schrodingers_cat_experiment_theta(theta):

    qc = QuantumCircuit(1)

    #your code start here
    qc.rx(theta, 0)

    #end of your code

    qc.measure_all()

    backend = AerSimulator()
    pm = generate_preset_pass_manager( backend = backend, optimization_level=3)
    qc_isa = pm.run(qc)

    # Circuit compile and run, shot = 1
    sampler = Sampler(mode=backend)
    counts = sampler.run([qc_isa], shots = 1).result()[0].data.meas.get_counts()

    measured_state = list(counts.keys())[0] if counts else '0' # bring measured result

    if measured_state == '0':
        cat_happy = True
    else:
        cat_happy = False

    return cat_happy, qc

# Submit your answer using the following code
grade_lab1_ex2(schrodingers_cat_experiment_theta)
```

⚡ Could not find a valid API server or the API servers are down right now.
Failed: None
Unable to grade your answer.

```
schrodingers_cat_experiment_theta(0)
```

⚡ (True, <qiskit.circuit.quantumcircuit.QuantumCircuit at 0x7b4a0bd13dd0>)

Now, use the widget below to check the cat's mood. Make sure `happy.png` and `grumpy.png` are in the same folder. (Image credit: James Weaver)

```
happy_img = Image.open('/content/happy.png') #note:download the picture from github repo and upload to google
grumpy_img = Image.open('/content/grumpy.png')
```

```
out = widgets.Output()
```

```
slider = widgets.FloatSlider(
    value=0,
    min=0,
    max=2*np.pi,
    step=0.01,
    description='θ',
    continuous_update=True
)
```

```
button = widgets.Button(
    description='Open the Box',
    button_style='success'
)
```

```
def on_button_click(b):
    with out:
        out.clear_output(wait=True) # clean output

        result = schrodingers_cat_experiment_theta(slider.value)[0]
```

```
if result==True:
    img = happy_img
    txt = "happy"
else:
    img = grumpy_img
    txt = "grumpy"

new_size = (400, 400)
resized_img = img.resize(new_size)

buf = io.BytesIO()
resized_img.save(buf, format='PNG')
buf.seek(0)
probability = int(np.cos(slider.value/2)**2 * 100)

display(f"The probability of cat is happy: {probability}%")
display(f"The observed cat is : {txt}")
display(widgets.Image(value=buf.read(), format='png'))

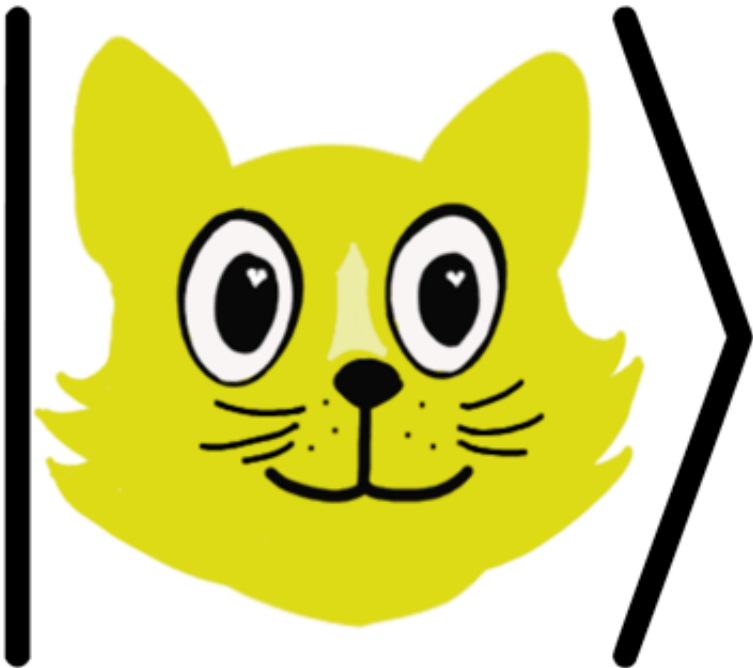
button.on_click(on_button_click)

display(slider, button, out)
```



Open the Box

'The probability of cat is happy: 50%'
'The observed cat is : happy'



Before we revisit the double-slit experiment, let's clarify what happens when we try to observe or "measure" a qubit, especially after applying a gate like $R_x(\theta)$. If you are familiar with quantum measurement, you can skip this part.

▼ Quantum Measurement details: Observing the Qubit

1. Qubit State Before Measurement: After $R_x(\theta)$ on $|0\rangle$, the state is a superposition:

$$|\psi\rangle = R_x(\theta)|0\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle - i \sin\left(\frac{\theta}{2}\right)|1\rangle = \alpha|0\rangle + \beta|1\rangle$$

where $|\alpha|^2 + |\beta|^2 = 1$.

2. Act of Measurement: Measurement in the computational basis ($\{|0\rangle, |1\rangle\}$) forces the qubit to choose one state. The superposition is not directly observable.

3. Probabilistic Outcomes: Probability of measuring $|0\rangle$: $P(0) = |\alpha|^2 = \cos^2\left(\frac{\theta}{2}\right)$ Probability of measuring $|1\rangle$: $P(1) = |\beta|^2 = \sin^2\left(\frac{\theta}{2}\right)$

4. State Collapse: Post-measurement, the state collapses:

- If '0' is measured $\rightarrow$ state becomes $|0\rangle$.
- If '1' is measured $\rightarrow$ state becomes $|1\rangle$. This "collapse" destroys the superposition.

Connection to Double-Slit: Detecting which slit a particle uses is a measurement. It collapses the particle's wave function to a definite path, destroying the superposition needed for interference fringes.

Let's revisit the double-slit experiment.

✓ Double-slit with Measurement

Exercise 3: Double-slit with a path detector

Your Goal: Construct a double-slit circuit that includes an intermediate measurement acting as a "which-path" detector. This will allow you to observe how acquiring information about the particle's path affects the final interference pattern.

Modify the double-slit circuit to include a "which-path" detector (a measurement after the first H-gate).

Setup:

- qr: 1 qubit ('q ').
- cr1: 1 classical bit ('c_detector') for path detection.
- cr2: 1 classical bit ('c_screen') for final measurement.
- ϕ : A Parameter for the phase gate.

Hint: Your circuit will have two measurements.

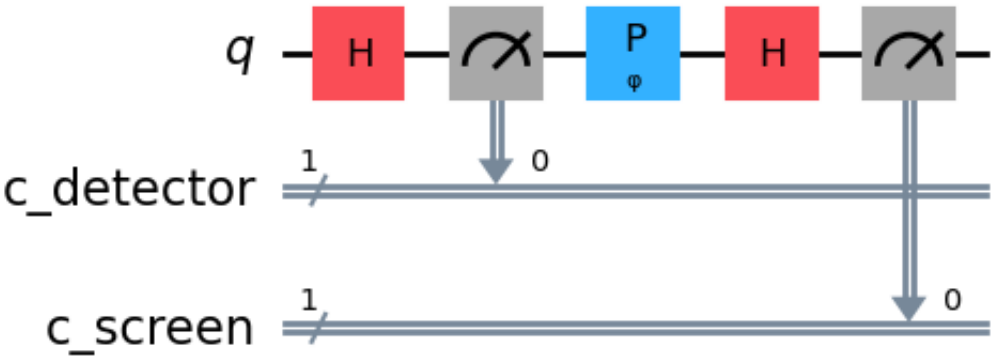
```
qr = QuantumRegister(1, name='q')
cr1 = ClassicalRegister(1, name='c_detector')
cr2 = ClassicalRegister(1, name='c_screen')
double_slit_with_detector = QuantumCircuit(qr, cr1, cr2)

phi = Parameter('phi')

#your code here
double_slit_with_detector.h(0)
double_slit_with_detector.measure(qr, cr1)
double_slit_with_detector.p(phi, qr)
double_slit_with_detector.h(0)
double_slit_with_detector.measure(qr, cr2)

#end of your code

double_slit_with_detector.draw('mpl')
```



```
# Submit your answer using the following code
grade_lab1_ex3(double_slit_with_detector)
```

Could not find a valid API server or the API servers are down right now.
Failed: None
Unable to grade your answer.

Now, repeat the heat map generation to see the new pattern with the detector.

Warning: This can take minutes

Resource limit

When running the code below on an actual backend, increasing the number of parameters or shots consumes more QPU time than might be expected. The current configuration (100 parameterized circuits × 1000 shots) uses less than 1 minute of QPU time, so please try to maintain these settings.

```

φ_lst = np.linspace(-3 * np.pi, 3 * np.pi, 100)
qc_isa = pm.run(double_slit_with_detector)

φ_hit = []
dist = sampler.run([(qc_isa, φ_lst)], shots=1000).result()[0].data.c_screen

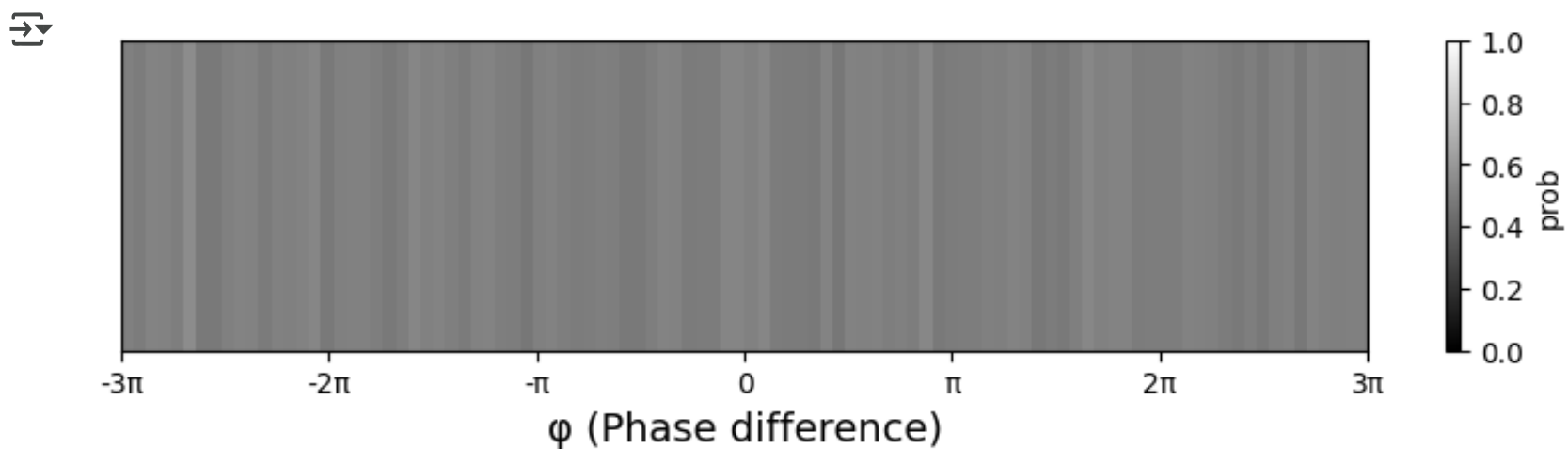
for i in range(len(φ_lst)):
    result = dist[i].get_counts()
    if '0' in result:
        φ_hit.append(result['0']/1000)
    else:
        φ_hit.append(0)

φ_hit_2d = np.array(φ_hit).reshape(1, -1)

plt.figure(figsize=(10, 2))
plt.imshow(φ_hit_2d, cmap='gray', aspect='auto', extent=[-3*np.pi, 3*np.pi, 0, 0.1], vmin=0, vmax=1)

plt.xlabel('φ (Phase difference)', fontsize=14)
plt.colorbar(label='prob')
plt.xticks(ticks=[-3 * np.pi, -2 * np.pi, -np.pi, 0, np.pi, 2 * np.pi, 3 * np.pi],
           labels=['-3π', '-2π', '-π', '0', 'π', '2π', '3π'])
plt.yticks([])
plt.show()

```

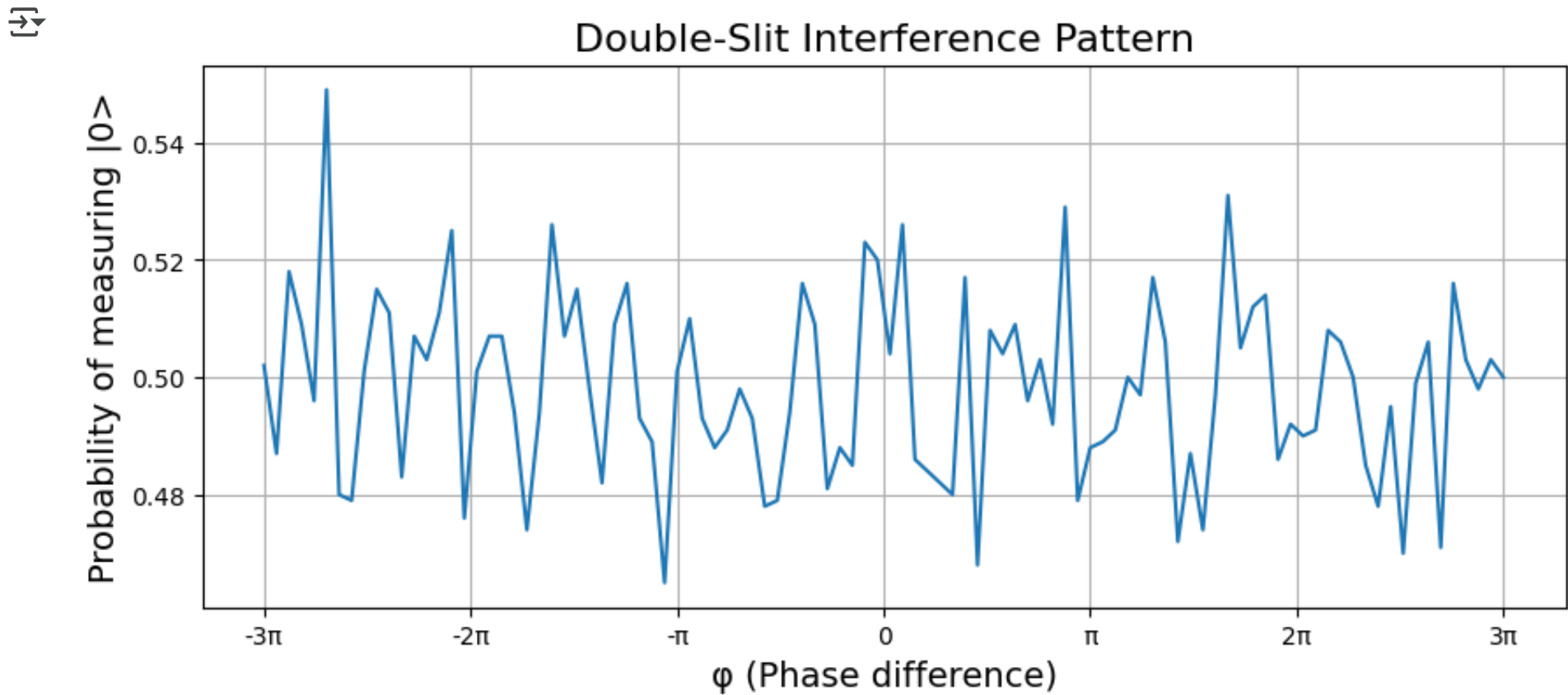


You should see a mostly gray pattern, indicating roughly 50% probability of measuring $|0\rangle$ regardless of ϕ . The interference fringes have vanished!

► Look into how detector works

So far, we've explored superposition, interference, and measurement. Now, let's dive into another profound quantum phenomenon: **entanglement**.

```
plt.figure(figsize=(10, 4))
plt.plot(phi_lst, phi_hit)
plt.xlabel('phi (Phase difference)', fontsize=14)
plt.ylabel('Probability of measuring |0>', fontsize=14)
plt.title('Double-Slit Interference Pattern', fontsize=16)
plt.xticks(ticks=[-3 * np.pi, -2 * np.pi, -np.pi, 0, np.pi, 2 * np.pi, 3 * np.pi],
          labels=['-3pi', '-2pi', '-pi', '0', 'pi', '2pi', '3pi'])
plt.grid(True)
plt.show()
```



Chapter 2: Entanglement

Quantum entanglement is a phenomenon where particles become interconnected, sharing a quantum state that links their properties, no matter how far apart they are. Measuring one particle's state instantly reveals information about the state of the other. Einstein called this “spooky action at a distance”.

In 1964, John Bell proposed Bell’s inequality to test if entangled particles behave classically. The CHSH experiment provided a concrete test. Quantum mechanics passed, defying classical expectations [3]. In 1997, quantum teleportation – transferring a quantum state without moving the particle – was achieved using entanglement [4].

With Qiskit, let's try the CHSH experiment and quantum teleportation.

2.1. The Curious Case of Alice, Bob, and the Unbeatable Game - CHSH Game

Alice and Bob are far apart and cannot communicate. A Referee gives them a challenge:

- 1. Referee picks two bits (x , y), sends x to Alice, y to Bob.
- 2. Alice and Bob instantly reply with their bits (a , b).
- 3. **Win Condition:** a XOR b == x AND y .

They want a pre-agreed strategy to maximize their average win rate.

Your Goal: Implement the *quantum* strategy for the CHSH game using Qiskit, simulate its performance, and compare it to the classical limit to understand the advantage provided by quantum entanglement.

For your in-depth study

For a more in-depth explanation and exploration of entanglement in action, including the CHSH game, you can refer to the Qiskit learning resource: [Entanglement in Action](#).

The Classical Limit: Why 75% is the Barrier

Let's first think classically. Since Alice and Bob can't communicate after receiving x and y , Alice's output a can only depend on x , and Bob's b only on y . They could pre-agree on a strategy, even a random one, but it boils down to local choices.

Winning condition means:

- Inputs (0,0), (0,1), or (1,0): Win if $a == b$.
- Input (1,1): Win if $a != b$.

Can you find a fixed strategy (like $a=0$, $b=0$ or $a=x$, $b=y$) that wins more than 3 out of the 4 possible input cases? Try it! You'll find the maximum average win rate is stubbornly stuck at **75%**, a fundamental limit of local realism.

The Quantum Strategy: Entanglement to the Rescue!

What if Alice and Bob prepared something special *before* the game? What if they shared a pair of **entangled qubits**? Imagine entanglement like two "magic" coins that are mysteriously linked. Even miles apart, measuring one instantly influences the *correlations* you'll find when measuring the other. They don't send messages faster than light, but their fates are intertwined.

Specifically, they share a [Bell state](#): $(|00\rangle + |11\rangle)/\sqrt{2}$. If they both measure their qubit in the *same* way, they always get the same result (both 0 or both 1).

The Quantum Strategy: Instead of outputting fixed bits, Alice and Bob use their input (x or y) to decide *how* to measure their shared qubit.

- Shared Resource: Alice holds qubit 0, Bob holds qubit 1 of their entangled pair.
- Measurement Choice (The Clever Part):
 - Alice (input x): Measures along angle 0 (standard Z basis) if $x=0$, or angle $\pi/2$ (X basis) if $x=1$.
 - Bob (input y): Measures along angle $-\pi/4$ if $y=0$, or angle $\pi/4$ if $y=1$, by using R_y gate.
- Output: They output the result of their respective measurements as a and b .

These specific angles are crucial! They exploit the unique correlations of the Bell state to maximize their chances of satisfying $a \oplus b == x \text{ AND } y$ across all input pairs.

Let's build the quantum circuit for this strategy.

Alice and Bob share an entangled qubit pair in the Bell state: $(|00\rangle + |11\rangle)/\sqrt{2}$. If measured the same way, they always get the same result.

Note

For detailed information on the quantum circuit's configuration, its operational flow, or the underlying principles, please refer to [this lesson](#) in the IBM Quantum Learning course, Basics of Quantum Information.

✓ Qiskit Implementation: Building the Quantum Circuit

Exercise 4: Quantum Circuit for CHSH Game

Your Goal: Define a function `create_chsh_circuit(x, y)` by constructing the quantum circuit that Alice and Bob will use for their quantum strategy in the CHSH game. This involves preparing an entangled Bell state and then applying measurement basis rotations based on their respective inputs.

Tasks:

- Task 1:** Create the Bell state $(|00\rangle + |11\rangle)/\sqrt{2}$.
- Task 2:** Implement Bob's rotation based on his input y .

```
def create_chsh_circuit(x, y):
    """Builds Qiskit circuit for Alice & Bob's quantum strategy."""
    qc = QuantumCircuit(2, 2, name=f'CHSH_{x}{y}') # 2 qubits, 2 classical bits

    # ---- TODO : Task 1 ---
    # Implement the gates to create the Bell state  $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ .
    qc.h(0)
    qc.cx(0, 1)

    # --- End of TODO ---
    qc.barrier()
    # Step 2a: Alice's measurement basis (X if x=1, Z if x=0)
    if x == 1:
        qc.h(0) # H for X-basis measurement

    ## --- TODO: Task 2 ----
    # Step 2b: Bob's measurement basis

    if y == 0: # Corrected angle for y=0
        qc.ry(-np.pi/4, 1)
    else: # Corrected angle for y=1
        qc.ry(np.pi/4, 1)

    # --- End of TODO ---
    qc.barrier()

    # Step 3: Measure
    qc.measure([0, 1], [0, 1]) # q0 to c0 (Alice), q1 to c1 (Bob) -> 'ba' format

    return qc
```

```
# Submit your answer using the following code
grade_lab1_ex4(create_chsh_circuit)
```

 Could not find a valid API server or the API servers are down right now.
 Failed: None
 Unable to grade your answer.

▼ Create Circuits for All Input Pairs

Now we use the function you created to generate circuits for all four (x,y) scenarios.

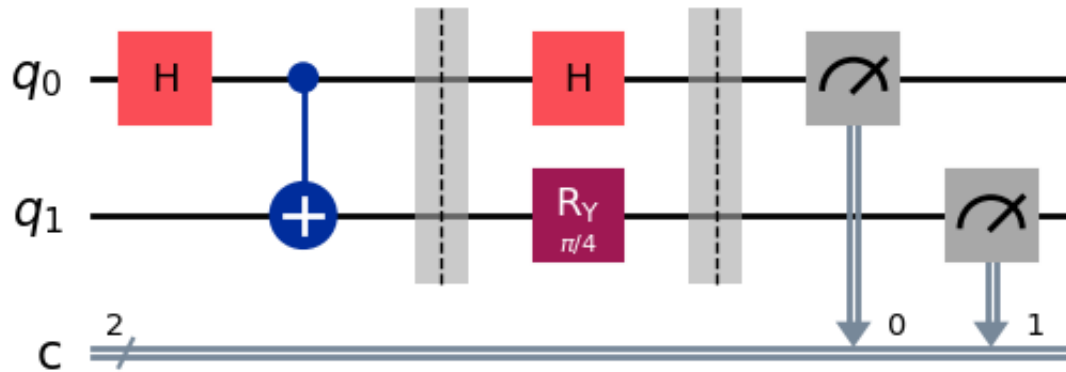
```

circuits = []
input_pairs = []
for x_in in [0, 1]:
    for y_in in [0, 1]:
        input_pairs.append((x_in, y_in))
        circuits.append(create_chsh_circuit(x_in, y_in))

print("Quantum circuit for inputs x=1, y=1 (Check your Exercises 1 & 2 implementation):")
if len(circuits) == 4:
    display(circuits[3].draw('mpl')) # (x,y) = (1,1)
else:
    print("Circuits not generated. Run previous cell after completing Exercises 1 & 2.")

```

➡ Quantum circuit for inputs x=1, y=1 (Check your Exercises 1 & 2 implementation):



✓ Simulation: Playing the Game Many Times

Real quantum computers are noisy and sometimes hard to access. Thankfully, we can simulate their behavior quite accurately for small systems like this one. We'll use Qiskit's `AerSimulator` to run our four circuits many times ("shots") and collect statistics on Alice and Bob's outputs (a and b).

```

# AerSimulator (if not already defined)
# backend = AerSimulator()
# Pass manager (if not already defined)
# pm = generate_preset_pass_manager(backend=backend, optimization_level=1)

SHOTS = 1024

print("Preparing circuits for the simulator...")
isa_qc_chsh = pm.run(circuits)

sampler_chsh = Sampler(mode=backend) # SamplerV2
job_chsh = sampler_chsh.run(isa_qc_chsh, shots=SHOTS)
results_chsh = job_chsh.result()

# SamplerV2: results_chsh[i].data.c.get_counts() where 'c' is the default name of classical register
counts_list = [results_chsh[i].data.c.get_counts() for i in range(len(circuits))]

print("\n--- Simulation Results (Counts) ---")
for i, (x, y) in enumerate(input_pairs):
    print(f"Inputs (x={x}, y={y}):")
    sorted_counts = dict(sorted(counts_list[i].items()))
    print(f"  Outcomes (ba): {sorted_counts}")

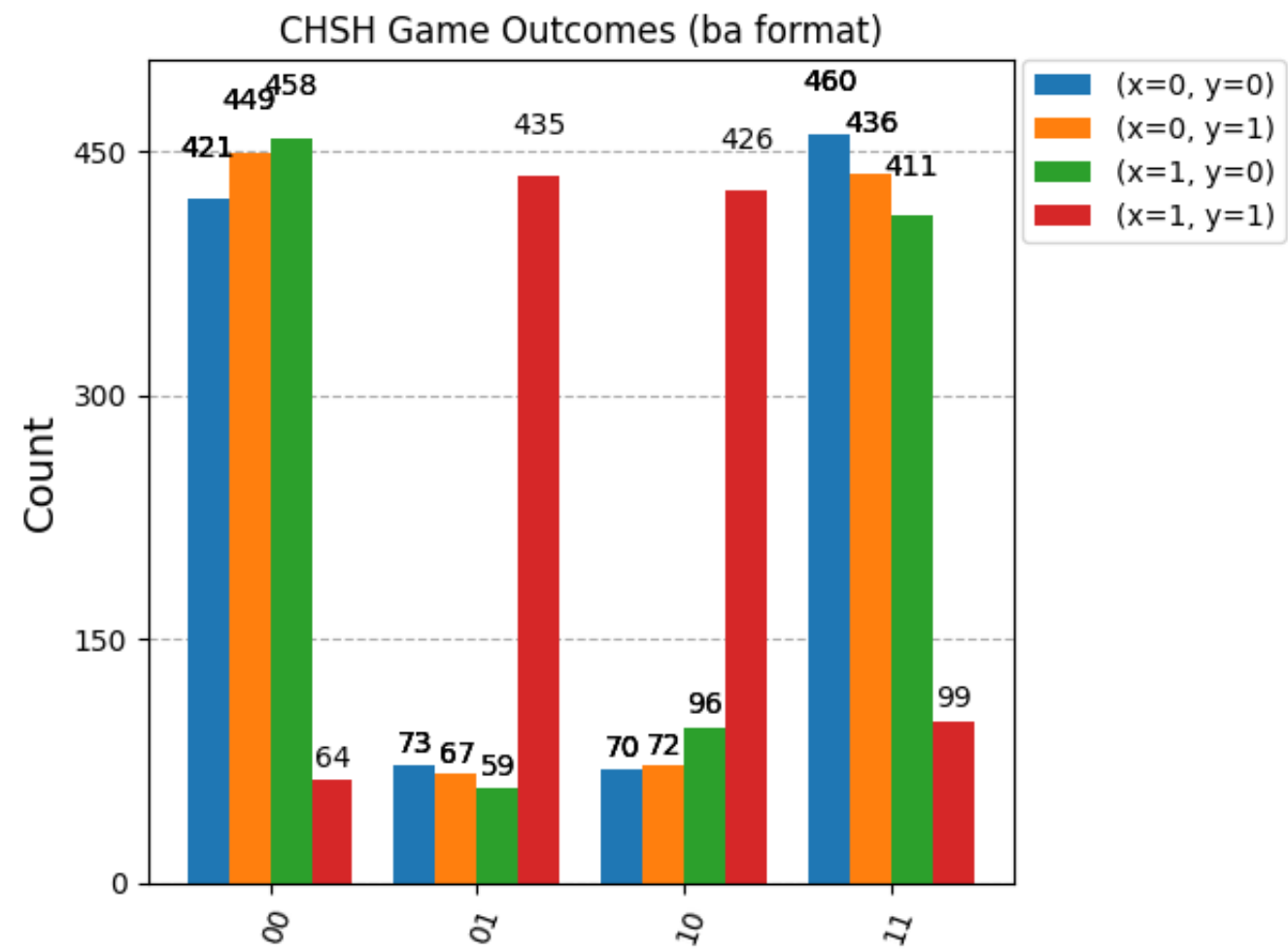
print("\nPlotting results...")
display(plot_histogram(counts_list,
                      legend=[f'(x={x}, y={y})' for x, y in input_pairs],
                      title='CHSH Game Outcomes (ba format)'))

```

➦ Preparing circuits for the simulator...

```
--- Simulation Results (Counts) ---
Inputs (x=0, y=0):
  Outcomes (ba): {'00': 421, '01': 73, '10': 70, '11': 460}
Inputs (x=0, y=1):
  Outcomes (ba): {'00': 449, '01': 67, '10': 72, '11': 436}
Inputs (x=1, y=0):
  Outcomes (ba): {'00': 458, '01': 59, '10': 96, '11': 411}
Inputs (x=1, y=1):
  Outcomes (ba): {'00': 64, '01': 435, '10': 426, '11': 99}
```

Plotting results...



✓ Analysis: Did They Beat the Classical Limit?

Now, the moment of truth! We need to analyze the simulation results (counts_list) to calculate the average win probability.

Recap:

- **Win Condition:** $a \oplus b == x \text{ AND } y$
- **Output Format:** Counts are for 'ba' strings.
 - $a \oplus b = 0$ for outcomes '00' (b=0, a=0) and '11' (b=1, a=1).
 - $a \oplus b = 1$ for outcomes '01' (b=0, a=1) and '10' (b=1, a=0).

Exercise 5: Analyze Circuit for CHSH Game

Your Goal: Calculate the win probability for Alice and Bob for each input case (x , y) based on the simulation results, and then determine their overall average win probability using the quantum strategy.

Tasks:

- **Task 1:** Determine the target $a \oplus b$ value for a win, given x and y .
- **Task 2:** Count shots (wins_for_this_case) satisfying the win condition for the current (x , y).

```

win_probabilities = {}
print("---- Calculating Win Probabilities ----")

for i, (x, y) in enumerate(input_pairs):
    counts = counts_list[i]

    # ---- TODO : Task 1 ----
    # Target (a XOR b) value for winning
    # The win condition is a XOR b == x AND y
    target_xor_result = 1 if (x and y) else 0

    # --- End of TODO ---

    wins_for_this_case = 0

    # ---- TODO : Task 2 ----
    # Calculate the total number of shots that satisfy the winning condition determined above. Check the 'target_xor'
    for outcome, count in counts.items():
        # outcome is in 'ba' format
        b = int(outcome[0])
        a = int(outcome[1])
        if (a ^ b) == target_xor_result:
            wins_for_this_case += count

    # --- End of TODO ---

    prob = wins_for_this_case / SHOTS if SHOTS > 0 else 0
    win_probabilities[(x, y)] = prob
    print(f"Inputs (x={x}, y={y}): Target (a XOR b) = {target_xor_result}. Win Probability = {prob:.4f}")

avg_win_prob = sum(win_probabilities.values()) / 4.0
P_win_quantum_theory = np.cos(np.pi / 8)**2 # ~0.8536
P_win_classical_limit = 0.75

print("\n--- Overall Performance ---")
print(f"Experimental Average Win Probability: {avg_win_prob:.4f}")
print(f"Theoretical Quantum Win Probability: {P_win_quantum_theory:.4f}")
print(f"Classical Limit Win Probability: {P_win_classical_limit:.4f}")

if avg_win_prob > P_win_classical_limit + 0.01: # Allow for small simulation variance
    print(f"\nSuccess! Your result ({avg_win_prob:.4f}) clearly beats the classical 75% limit!")
    print(f"It's likely close to the theoretical quantum prediction of {P_win_quantum_theory:.4f}.")
elif avg_win_prob > P_win_classical_limit - 0.02 : # Could be noise or minor error
    print(f"\nClose, but no cigar? Your result ({avg_win_prob:.4f}) is around the classical limit ({P_win_classical_limit:.4f}).")
    print("Check your solutions for Exercises 1-4 carefully, especially the win counting logic in Ex 4.")
else:
    print(f"\nHmm, the result ({avg_win_prob:.4f}) is unexpectedly low, even below the classical limit.")
    print("There might be an error in Exercises 1-4. Please review your circuit and analysis code.")

➡ ---- Calculating Win Probabilities ----
Inputs (x=0, y=0): Target (a XOR b) = 0. Win Probability = 0.8604
Inputs (x=0, y=1): Target (a XOR b) = 0. Win Probability = 0.8643
Inputs (x=1, y=0): Target (a XOR b) = 0. Win Probability = 0.8486
Inputs (x=1, y=1): Target (a XOR b) = 1. Win Probability = 0.8408

---- Overall Performance ----
Experimental Average Win Probability: 0.8535
Theoretical Quantum Win Probability: 0.8536
Classical Limit Win Probability: 0.7500

Success! Your result (0.8535) clearly beats the classical 75% limit!
It's likely close to the theoretical quantum prediction of 0.8536.

# Submit your answer using the following code
grade_lab1_ex5(counts_list, avg_win_prob)

```

Conclusion: The Quantum Edge

So, what did we find? If you successfully completed the exercises, your simulation should clearly show that Alice and Bob, using their shared entangled qubits and clever measurements, achieved an average win rate **significantly** higher than the 75% classical limit, likely approaching the theoretical quantum maximum of around **85.4%**.

This isn't just a mathematical trick; it reflects how nature works at its most fundamental level. Entanglement allows for correlations between distant particles that are stronger than any classical theory can explain. This "spooky action at a distance" doesn't allow faster-than-light communication, but it underpins many quantum technologies.

Discussion Point: Based on these results, what does the CHSH game demonstrate about the nature of reality compared to classical intuition? Can you think of any potential applications where these stronger-than-classical correlations might be useful?

✓ 2.2. Quantum Teleportation - Sending Secrets with Spooky Action

Imagine Alice has a qubit (q_0) in a specific, delicate quantum state ($|\psi\rangle$). She wants to send this exact state to Bob, who is far away. She can't simply physically send the qubit (maybe it's too fragile, or she needs the original particle). Can she somehow transfer *only the state* itself?

Classically, this seems impossible. If Alice measures her qubit to see what state it's in, the measurement itself will likely collapse the superposition and destroy the original state.

Enter **Quantum Teleportation**! It's a remarkable protocol that uses **quantum entanglement** and **classical communication** to transfer an unknown quantum state from one qubit to another.

Disclaimer: We're teleporting the *state*, not the physical particle! No people are being beamed up here.

Your Goal: In this lab, you will implement the quantum teleportation protocol using Qiskit. You'll complete the code to prepare the necessary quantum resources, perform the key steps, and verify that Bob successfully receives Alice's quantum state.

The Teleportation Protocol: Step-by-Step

The protocol requires three qubits and two classical bits:

- **q0** : Alice's qubit, initially in the state $|\psi\rangle$ we want to teleport.
- **q1** : Alice's half of a shared entangled pair.
- **q2** : Bob's half of the shared entangled pair.
- **c0, c1** : Classical bits to store Alice's measurement results.

Here's the plan:

1. **(Optional) Prepare Alice's state $|\psi\rangle$ on q0** . (We'll create a specific state like $|+\rangle$ for verification).
2. **Create entanglement**: Generate a Bell pair between q1 and q2 .
3. **Alice's operations**: Alice performs a "Bell measurement" on her two qubits (q0 and q1) and stores the classical results in c0 and c1 .
4. **Classical communication**: Alice sends her two classical bits (c0 , c1) to Bob.
5. **Bob's corrections**: Bob applies specific quantum gates (X and/or Z) to his qubit (q2), conditioned on the values of c0 and c1 he received.

If everything is done correctly, Bob's qubit q2 will end up in the state $|\psi\rangle$, the original state of Alice's q0 !

For your in-depth study

For a more in-depth explanation and exploration of quantum teleportation, you can refer to the IBM Quantum Learning resources: [Quantum Teleportation](#) and [Entanglement in Action](#) by John Watrous. These are part of his [Basics of Quantum Information](#) course.

Tool: Qiskit Dynamic Circuits

For Bob's actions depending on Alice's measurements, we use dynamic circuits. This allows classical information from mid-circuit measurements to influence later quantum operations. This capability is crucial for protocols like teleportation and for creating efficient long-range entanglement on quantum hardware, as explored in recent research (e.g., Bäumer et al., PRX QUANTUM 5, 030339 (2024)).

Reference:

- [Qiskit documentation on dynamic circuits and conditional operations](#)
- [Efficient Long-Range Entanglement Using Dynamic Circuits](#)

Key Concepts for Teleportation:

1. Mid-Circuit Measurement:
 - In Qiskit, you can measure a qubit and store the result in a classical bit at any point in the circuit, not just at the end. This is crucial for Alice to measure her qubits (q0 and q1) and obtain classical bits c0 and c1 .
 - The `QuantumCircuit.measure(qubit, classical_bit)` instruction is used.
2. Conditional Operations (if_test):
 - After Alice's measurements, Bob needs to apply corrective gates to his qubit (q2) based on the classical bits (c0 , c1) he receives.
 - Qiskit allows gates to be applied conditionally based on the state of classical bits.
 - The `QuantumCircuit.if_test((classical_bit, value), true_circuit, false_circuit=None)` method can be used to apply gates only if a certain classical bit has a specific value (0 or 1).
 - For teleportation, Bob will use these conditional operations:
 - If c1 (here `cr_alice_tele[1]`) is 1, apply an X gate to q2 .
 - If c0 (here `cr_alice_tele[0]`) is 1, apply a Z gate to q2 .

These features enable the classical communication aspect of the teleportation protocol to be directly implemented and simulated within a single quantum circuit description.

Note: Please remember the sequence of conditional gates! In Quantum Computing $AB \neq BA$ so the order is matter.

✓ Building the Teleportation Circuit in Qiskit

Exercise 6: Quantum Teleportation

Your Goal: Construct the complete quantum circuit for teleporting an unknown quantum state from Alice's message qubit to Bob's qubit, utilizing an entangled Bell pair and mid-circuit measurements with conditional operation.

Tasks:

- **Step 1:** Create the shared entangled Bell pair $(|00\rangle + |11\rangle)/\sqrt{2}$ between q1 and q2.
- **Step 2:** Implement Alice's Bell measurement gates (before actual measurement).
- **Step 3:** Implement Bob's conditional correction gates based on Alice's measurement results.

```
# Define quantum and classical registers
qr_tele = QuantumRegister(3, name='q')
cr_alice_tele = ClassicalRegister(2, name='c_alice') # For Alice's measurements

# For verification with statevector, we don't measure Bob's final qubit in this circuit.
# If we were to run on hardware and verify by counts, we'd add a classical bit for Bob.
teleport_qc = QuantumCircuit(qr_tele, cr_alice_tele, name='Teleportation')

# Prepare Alice's message state  $|\psi\rangle = |+\rangle$  on q0
teleport_qc.h(qr_tele[0])
teleport_qc.barrier()

# ---- TODO : Task 1 ---
# Step 1: Create Bell pair between q1 (Alice) and q2 (Bob)
teleport_qc.h(qr_tele[1])
teleport_qc.cx(qr_tele[1], qr_tele[2])

# --- End of TODO ---
teleport_qc.barrier()

# ---- TODO : Task 2 ---
# Step 2: Alice's Bell Measurement (gates part)
teleport_qc.cx(qr_tele[0], qr_tele[1])
teleport_qc.h(qr_tele[0])

# --- End of TODO ---
teleport_qc.barrier()

# Alice measures her qubits q0 and q1
teleport_qc.measure(qr_tele[0], cr_alice_tele[0]) # q0 -> c0
teleport_qc.measure(qr_tele[1], cr_alice_tele[1]) # q1 -> c1
teleport_qc.barrier()

# ---- TODO : Task 3 ---
# Step 3: Bob's Conditional Corrections on q2
# IMPORTANT: .c_if() on gates like XGate() no longer works directly as in older Qiskit versions.
# The recommended method in Qiskit 1.0+ is to use the new `if_test` context manager.
## teleport_qc.if_test((cr_alice_tele[1], 1), teleport_qc.x(qr_tele[2]))
# Create a small circuit for the X gate to be applied conditionally
x_gate_circuit = QuantumCircuit(1)
x_gate_circuit.x(0)

# Apply X gate to q2 if cr_alice_tele[1] is 1
teleport_qc.if_test((cr_alice_tele[1], 1), x_gate_circuit, [qr_tele[2]], [])

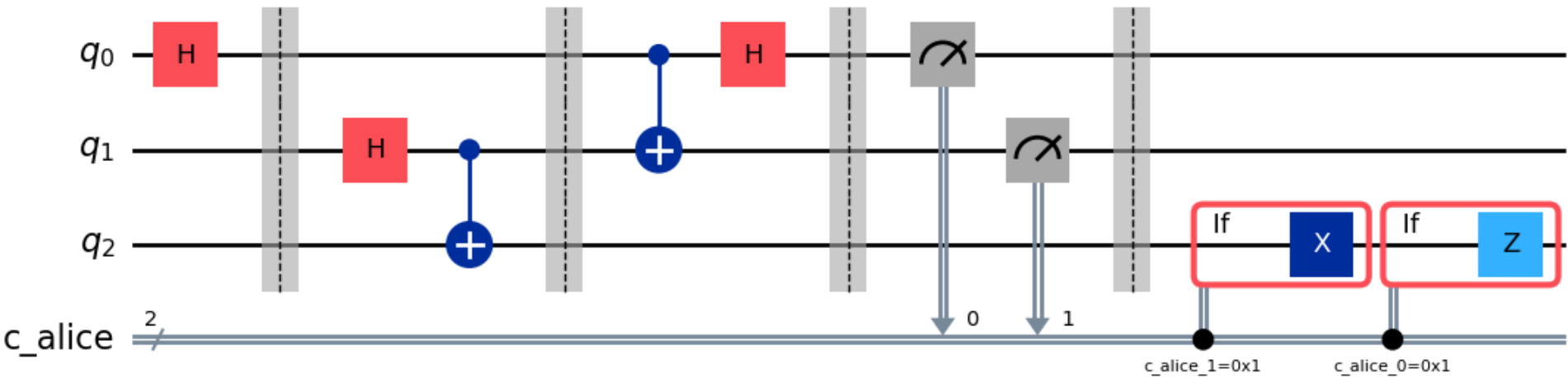
# Create a small circuit for the Z gate to be applied conditionally
z_gate_circuit = QuantumCircuit(1)
z_gate_circuit.z(0)

# Apply Z gate to q2 if cr_alice_tele[0] is 1
teleport_qc.if_test((cr_alice_tele[0], 1), z_gate_circuit, [qr_tele[2]], [])

# --- End of TODO ---
```

```
print("Full Teleportation Circuit (Check your Exercises 1, 2, 3):")
display(teleport_qc.draw('mpl'))
```

Full Teleportation Circuit (Check your Exercises 1, 2, 3):



```
# Submit your answer using the following code
grade_lab1_ex6(teleport_qc)
```

Could not find a valid API server or the API servers are down right now.
Failed: None
Unable to grade your answer.

Simulation and Verification

How do we verify that the teleportation worked? We can't directly 'see' the state of Bob's qubit after the protocol. However, since we *prepared* Alice's initial state $|\psi\rangle$ (we chose $|+\rangle$), we can use a special type of simulation to check if Bob's qubit q_2 ended up in that same state.

We'll use `AerSimulator` with `save_statevector` to check if Bob's qubit q_2 ends up in Alice's original state ($|+\rangle$). This simulator calculates the final quantum state vector.

Exercise 7 - No Grading: Analyze result of Quantum Teleportation

Your Goal: Verify the successful teleportation of Alice's quantum state to Bob's qubit by simulating the circuit and visualizing the final state of Bob's qubit.

Task:

- Extract the final statevector and use `plot_bloch_multivector` to visualize Bob's qubit (q_2). Compare to Alice's initial state (q_0).

```
%matplotlib inline
from qiskit.visualization import plot_bloch_multivector

# Use Statevector Simulator
print("Using statevector simulator...")
sv_simulator = AerSimulator(method='statevector') # Explicitly set method for clarity
teleport_qc_sv = teleport_qc.copy() # Work with a copy for statevector simulation
teleport_qc_sv.save_statevector() # Save statevector at the end

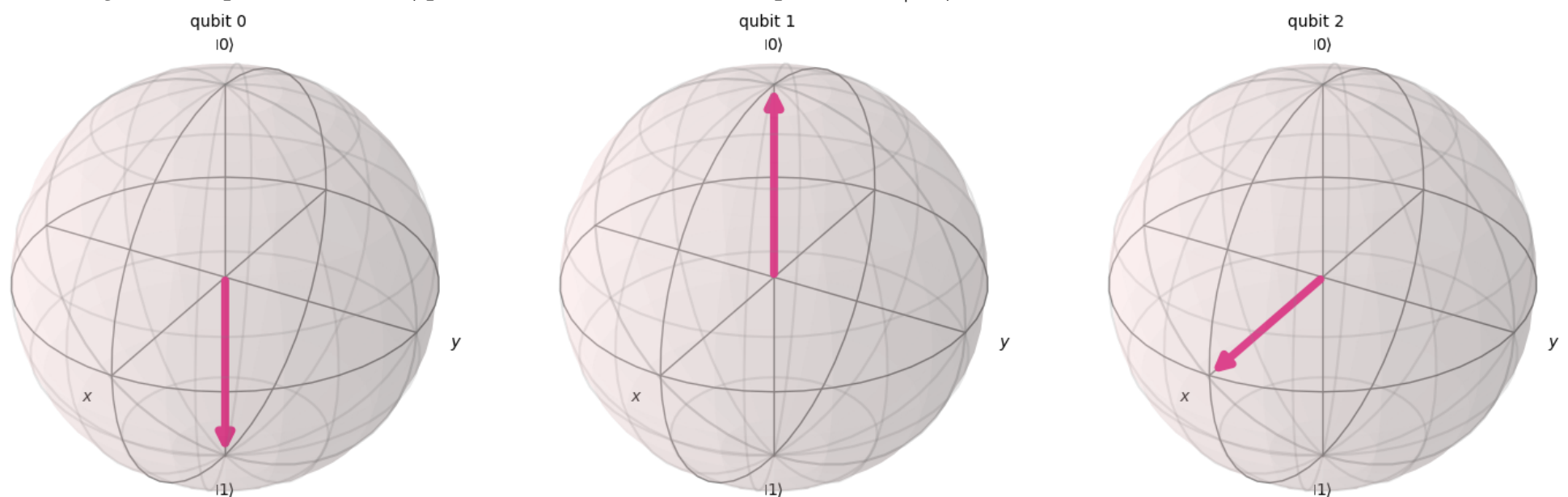
print("Running statevector simulation...")
job_sv = sv_simulator.run(teleport_qc_sv) # shots=1 is default for statevector
result_sv = job_sv.result()

if result_sv.success:
    print("Simulation successful.")
    final_statevector = result_sv.get_statevector()
    print("Statevector retrieved successfully.")
    print("\nVisualizing final qubit states (q2 should match initial q0 state |+>):")
    # q0 was |+> (points along +X). After teleportation, q2 should be |+>.
    # q0 and q1 states are after Alice's measurement, so they'll be collapsed.
    #display("TODO") #TODO, use plot_bloch_multivector to plot final_state
    display(plot_bloch_multivector(final_statevector))

else:
    print(f"Statevector simulation failed! Status: {result_sv.status}")
```

```
⇒ Using statevector simulator...
Running statevector simulation...
Simulation successful.
Statevector retrieved successfully.
```

Visualizing final qubit states (q2 should match initial q0 state $|+\rangle$):



Interpreting the Results

If Exercise 6 was correct, you should see:

- q0 (Alice's original): Random state (collapsed by measurement).
- q1 (Alice's entangled): Random state (collapsed by measurement).
- q2 (Bob's entangled): Bloch sphere vector pointing along the **positive X-axis**. This is the $|+\rangle$ state, matching Alice's initial q0!

Success! The state $|\psi\rangle = |+\rangle$ was teleported.

✓ Conclusion: The Magic of Entanglement and Information

You have successfully implemented the quantum teleportation protocol!

Key Takeaways:

- Teleportation transfers an unknown quantum *state* using shared entanglement and classical communication.
- It doesn't transfer the physical particle itself.
- The original state on Alice's qubit is destroyed (consistent with the No-Cloning Theorem).
- Requires classical communication (Alice's measurement results to Bob), so no faster-than-light information transfer.

Quantum teleportation is foundational for quantum communication and computation.

Discussion Point: Why can't Alice just measure q_0 (e.g., in the Z basis then the X basis) and send results to Bob for reconstruction?

What's different about teleportation?

```
#Check your submission status with the code below
from qc_grader.grader.grade import check_lab_completion_status
check_lab_completion_status("qgss_2025")
```

```
↔ Could not find a valid API server or the API servers are down right now.
Failed: Invalid URL 'None': No scheme supplied. Perhaps you meant https://None?
[]
```

✓ (Bonus Challenge) Teleportation on Real Quantum Hardware

We've seen teleportation work perfectly on a simulator. Now, let's try it on a **real IBM Quantum backend**! This is where the true challenges and excitement of quantum computing lie. Real devices have noise and limited qubit connectivity.

Dynamic Circuits are being upgraded!

There is currently an update going on for dynamic circuits on the real hardware, and you are one of the first people to be able to test the new version! However, this early access comes with some caveats:

- 1. We need to set some additional lines of codes for them to activate the early access (see below)
- 2. This early access doesn't work when using Session or Batch with a with... (e.g. with Batch(...)). You can, however, pass one directly to the primitive (e.g. Sampler(mode=batch)).
- 3. You cannot use a ClassicalRegister of more than 32 bits in an if_test condition. You can, however, use multiple ClassicalRegisters, each of ≤ 32 bits.
- 4. No nested conditional.
- 5. Cannot have measure or reset inside a conditional block.

Two steps are needed to use dynamic circuits on the real hardware:

Step 1 you need to add the instruction to the hardware you use:

```
# (1) Patch the backend target with IfElseOp
from qiskit.circuit import IfElseOp
backend.target.add_instruction(IfElseOp, name="if_else")
```

Step 2 when submitting the jobs you need to enable the experimental options:

```
#(2) Submit the job with the experimental option
sampler.options.experimental = {"execution_path" : "gen3-experimental"}
```

Dynamic Circuits: A Key for Real Hardware

The teleportation protocol inherently requires **dynamic circuits**: Alice measures her qubits, and Bob *classically communicates* these

results to decide which correction gates to apply to his qubit. Qiskit enables these "mid-circuit measurements" and "classical feed-forward" operations.

Recent research, such as the work by Bäumer et al. in "Efficient Long-Range Entanglement Using Dynamic Circuits" (PRX QUANTUM 5, 030339 (2024)), demonstrates the power of dynamic circuits. They show that for tasks like CNOT gate teleportation and preparing GHZ states over long distances, dynamic circuits can outperform their purely unitary counterparts on large-scale devices by keeping the quantum circuit depth constant regardless of the distance.

For our teleportation protocol, dynamic circuits allow Alice's measurement outcomes to directly control Bob's gate applications in real time on the quantum processor. Accessing IBM Quantum backends is done through the **IBM Cloud**. You'll need an IBM Cloud account with Quantum services enabled.

The Challenge: – How Far Can You Send It?

Your Goal: Successfully teleport a quantum state (e.g., $|+\rangle$) from 0th qubit (Alice's message) to another qubit (Bob's) on a real IBM Quantum backend, ideally choosing qubits that are not directly connected on the chip by using Qiskit Dynamic Circuits. Observe and report the fidelity of the teleportation. Try to improve it!

Basic steps for a real backend run:

You can find more detailed explanations in lab0, but here we give you an abstracted intro.

1. Set up IBM Quantum Access (via IBM Cloud):

- If you don't have one, create an IBM Cloud account and enable access to IBM Quantum services.
- Get your API token from the IBM Quantum dashboard on IBM Cloud.
- Use `QiskitRuntimeService` to save your account and list available backends.

```
from qiskit_ibm_runtime import QiskitRuntimeService
QiskitRuntimeService.save_account(channel="ibm_quantum_platform", token="YOUR_IBM_CLOUD_API_TOKEN", insecure=True)
service = QiskitRuntimeService(name="qgss-2025")
print(service.backends())
```

2. Choose a Backend:

- Select an available backend from the list. Selecting the [least_busy](#) QPU is also a good choice.
- Examine its coupling map to find suitable qubits. For this challenge, try to pick qubits for Alice's message (`q_A_msg`) and Bob's final qubit (`q_B_ent`) that are "far apart" or not directly connected. Alice's entangled qubit (`q_A_ent`) should ideally be connectable to both `q_A_msg` (for her Bell measurement) and `q_B_ent` (for creating the initial Bell pair).

```
#backend_name = "ibm_your_chosen_backend" # e.g., "ibm_brisbane" or a simulator like "ibmq_qasm_simulator"
backend = service.least_busy()
# print(f"Coupling map: {backend.configuration().coupling_map}")
# print(f"Number of qubits: {backend.configuration().n_qubits}")
# from qiskit.visualization import plot_gate_map # if needed
# display(plot_gate_map(backend)) # Visualizes connectivity
```

You can keep most of the code the same. To use a real quantum processing unit (QPU) instead of a simulator, simply pass the real backend object to the `Sampler`. However, it's crucial to transpile your circuit using a `PassManager`, especially when targeting actual quantum hardware.

Tips

- Technical Reference for Long-Range Entanglement: For a deeper technical dive into creating entanglement over distances on IBM hardware, including techniques that might inspire your qubit selection or circuit optimization, refer to the tutorial [Efficiently generating long-range entanglement](#).
- Simulating Larger Circuits (20+ Qubits):
 - If you're designing or testing teleportation schemes that scale to 20 qubits or more for the total circuit size (including Alice's message, her ancilla, Bob's qubit, and any intermediary qubits for routing or advanced teleportation schemes), simulating the

full statevector can become computationally intensive.

- For such larger circuits, if your teleportation circuit (or components of it, like Bell state creation and Bell measurements) can be constructed using only **Clifford** gates (H, S, X, Y, Z, CNOT, SWAP, and their combinations), you can use a more efficient simulator.
- Recommendation: Use `AerSimulator(method="stabilizer")` . The stabilizer simulator is optimized for Clifford circuits and can handle much larger qubit counts efficiently.

Good luck pushing the limits of quantum state transfer! Share your try and result with participants and discuss how you can send qubits "farther".

References

1. Young, Thomas (1804) I. The Bakerian Lecture. Experiments and calculations relative to physical optics. Phil. Trans. R. Soc. 941–16.
2. Fage, Arthur and Johansen, F. C. (1927). On the flow of air behind an inclined flat plate of infinite span. Proc. R. Soc. Lond. A116 170–197.
3. Clauser, J. F., Horne, M. A., Shimony, A., & Holt, R. A. (1969). Proposed Experiment to Test Local Hidden-Variable Theories. Phys. Rev. Lett., 23(15), 880–884.
4. Bouwmeester, D., Pan, J.-W., Mattle, K., Eibl, M., Weinfurter, H., & Zeilinger, A. (1997). Experimental quantum teleportation. Nature, 390(6660), 575–579.

Additional information

Created by: Sophy Shin, James Weaver

Advised by: Marcel Pfaffhauser, Jessie Yu, Junye Huang, Borja Peropadre

Reviewed by: Meltem Tolunay, Abby Cross

Version: 1.1.1